COMPLETE PROGRAMMING GUIDE

The Complete Java Mastery Guide

From Beginner to Advanced — Core Java, OOP, Collections, DSA & More

Comprehensive Java Programming Reference

Data Structures & Algorithms Included

2026 Edition

Table of Contents

Chapter 1: Introduction to Java

1.1 What is Java?

1.2 Setting Up the Environment

1.3 Your First Java Program

1.4 How Java Works

Chapter 2: Java Basics

2.1 Variables and Data Types

2.2 Operators

2.3 Type Casting

2.4 User Input

Chapter 3: Control Flow Statements

3.1 If-Else Statements

3.2 Switch Statement

3.3 Loops (for, while, do-while)

3.4 Break and Continue

Chapter 4: Object-Oriented Programming

4.1 Classes and Objects

4.2 Constructors

4.3 Encapsulation

4.4 Inheritance

4.5 Polymorphism

4.6 Abstraction

4.7 Interfaces

Chapter 5: Arrays and Strings

5.1 Arrays

5.2 Multi-Dimensional Arrays

5.3 String Class

5.4 StringBuilder and StringBuffer

Chapter 6: Exception Handling

6.1 Types of Exceptions

6.2 Try-Catch-Finally

6.3 Throw and Throws

6.4 Custom Exceptions

Chapter 7: Generics

7.1 Generic Classes

7.2 Generic Methods

7.3 Bounded Type Parameters

7.4 Wildcards

Chapter 8: Lambda Expressions and Functional Interfaces

8.1 Functional Interfaces

8.2 Lambda Syntax

8.3 Method References

Chapter 9: Streams API

9.1 Creating Streams

9.2 Intermediate Operations

9.3 Terminal Operations

9.4 Collectors

Chapter 10: Java Collections Framework

10.1 Collection Hierarchy

10.2 List Interface

10.3 Set Interface

10.4 Queue Interface

10.5 Map Interface

10.6 Choosing the Right Collection

Chapter 11: Data Structures

11.1 Linked List

11.2 Stack

11.3 Queue

11.4 Binary Trees

11.5 Binary Search Tree

11.6 Heaps and Priority Queues

11.7 Hash Tables

11.8 Graphs

Chapter 12: Algorithms

12.1 Sorting Algorithms

12.2 Searching Algorithms

12.3 Recursion

12.4 Dynamic Programming

Chapter 13: Multithreading and Concurrency

13.1 Creating Threads

13.2 Thread Lifecycle

13.3 Synchronization

13.4 Executors Framework

Chapter 14: File I/O and Serialization

14.1 File Class

14.2 Reading and Writing Files

14.3 Serialization

Chapter 15: Best Practices and Design Patterns

15.1 Coding Standards

15.2 Common Design Patterns

15.3 Memory Management

Chapter 1

Introduction to Java

1.1 What is Java?

> **Definition**
>
> **Java** is a high-level, object-oriented, platform-independent programming language developed by Sun Microsystems (now Oracle) in 1995. It follows the principle of **"Write Once, Run Anywhere" (WORA)**, meaning compiled Java code can run on any platform that has a Java Virtual Machine (JVM).

Key Characteristics of Java

Table 1-1: Java Language Characteristics

Characteristic	Description
Object-Oriented	Everything in Java revolves around classes and objects
Platform Independent	Bytecode runs on any OS with JVM support
Compiled & Interpreted	Source code compiles to bytecode, then interpreted by JVM
Strongly Typed	Every variable must have a declared type
Memory Managed	Automatic garbage collection manages memory
Secure	Bytecode verification, no direct memory access
Multithreaded	Built-in support for concurrent programming

History of Java

Java was created by **James Gosling** and his team at Sun Microsystems. Originally named "Oak," it was renamed Java in 1995. The language has evolved significantly:

- **Java 8 (2014)**: Major release with Lambda expressions, Streams API, new Date/Time API
- **Java 11 (2018)**: LTS release with var keyword, new HTTP client
- **Java 17 (2021)**: LTS with sealed classes, pattern matching
- **Java 21 (2023)**: Virtual threads, sequenced collections

1.2 Setting Up the Environment

Installing JDK

To develop Java applications, you need the **Java Development Kit (JDK)**:

1. Download JDK from Oracle or use OpenJDK distributions
2. Set the `JAVA_HOME` environment variable
3. Add `%JAVA_HOME%\bin` to your system PATH

Popular IDEs

Table 1-2: Java Integrated Development Environments

IDE	Developer	Key Features
IntelliJ IDEA	JetBrains	Smart code completion, refactoring, debugging
Eclipse	Eclipse Foundation	Free, extensible with plugins
VS Code	Microsoft	Lightweight, extensions support
NetBeans	Apache	Built-in GUI builder, Maven support

1.3 Your First Java Program

> **Structure of a Java Program**
>
> Every Java program has at least one **class** containing the `main` method — the entry point where execution begins.

```java
// HelloWorld.java
// This is a single-line comment

/*
 * This is a multi-line comment
 * Used for detailed documentation
 */

/**
 * This is a Javadoc comment
 * Used for generating API documentation
 */

public class HelloWorld {
    // The main method - entry point of the program
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Output:

```
Hello, World!
```

Understanding the Components

```
public class HelloWorld {
    //   ^     ^
    //   |     +-- Class name (must match filename)
    //   +-- Access modifier (public = accessible everywhere)

    public static void main(String[] args) {
    // ^      ^      ^       ^
     // |        |        |         +-- Parameter: array of command-line
arguments
    // |      |      +-- Return type (void = no return value)
    // |      +-- Belongs to class, not instance
    // +-- Accessible from anywhere

        System.out.println("Hello, World!");
        // ^         ^        ^
        // |         |        +-- Method name
        // |         +-- Output stream (console)
        // +-- System class (provides system resources)
    }
}
```

Compiling and Running Java Code

```
// Step 1: Compile the source code
// Command: javac HelloWorld.java
// Result: HelloWorld.class (bytecode file)

// Step 2: Run the compiled program
// Command: java HelloWorld
// Result: Hello, World!
```

Important Notes

- The filename must match the public class name (HelloWorld.java)
- Java is case-sensitive: `HelloWorld` is different from `helloworld`
- Every statement ends with a semicolon `;`
- Code blocks are enclosed in curly braces `{ }`

1.4 How Java Works

Java uses a unique two-step execution process that enables platform independence:

The Java Compilation Process

1. **Source Code (.java)**: Human-readable Java code written by the programmer
2. **Compiler (javac)**: Converts source code into bytecode
3. **Bytecode (.class)**: Platform-independent intermediate code
4. **JVM (Java Virtual Machine)**: Interprets/JIT compiles bytecode to native machine code
5. **Native Code**: Machine-specific instructions executed by the OS

> **Java Virtual Machine (JVM)**
>
> The **JVM** is an abstract computing machine that enables a computer to run Java programs. It provides runtime environment, memory management, security, and platform independence. Each operating system has its own JVM implementation.

> **Java Runtime Environment (JRE)**
>
> The **JRE** contains the JVM plus core libraries and supporting files needed to run Java applications. It does not include development tools like the compiler.

> **Java Development Kit (JDK)**
>
> The **JDK** is a superset of JRE that includes development tools: compiler (javac), debugger, archiver (jar), documentation generator (javadoc), and more.

```java
// Understanding package declaration
package com.example.myapp;

// Packages organize related classes
// Directory structure: com/example/myapp/Hello.java

import java.util.Scanner;  // Import a specific class
import java.util.*;         // Import all classes in a package

public class Hello {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();
        System.out.println("Hello, " + name + "!");
        scanner.close();
    }
}
```

Output:

```
Enter your name: Alice Hello, Alice!
```

Chapter 2

Java Basics

2.1 Variables and Data Types

> **Definition: Variable**
>
> A **variable** is a named memory location that stores data which can be modified during program execution. In Java, every variable must be declared with a specific data type before it can be used.

> **Definition: Data Type**
>
> A **data type** specifies the type of data a variable can hold, the amount of memory allocated, and the operations that can be performed on that data.

Declaring Variables

```
// Syntax: dataType variableName = value;

int age = 25;                    // Integer
double salary = 50000.50;        // Decimal number
char grade = 'A';                // Single character
boolean isActive = true;         // True or false
String name = "John Doe";              // Text (String is a class, not
primitive)

// Multiple declarations on one line
int x = 10, y = 20, z = 30;

// Declaring without initialization
int count;      // Contains default value 0
String message; // Contains default value null
```

Primitive Data Types

Java has 8 primitive data types that store simple values directly in memory:

Table 2-1: Java Primitive Data Types

Type	Size	Range	Default Value	Example
`byte`	1 byte	-128 to 127	0	`byte b = 100;`
`short`	2 bytes	-32,768 to 32,767	0	`short s = 1000;`
`int`	4 bytes	-2,147,483,648 to 2,147,483,647	0	`int i = 100000;`
`long`	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	0L	`long l = 10000000000L;`
`float`	4 bytes	~7 decimal digits precision	0.0f	`float f = 3.14f;`
`double`	8 bytes	~15 decimal digits precision	0.0	`double d = 3.14159;`
`char`	2 bytes	0 to 65,535 (Unicode)	'\u0000'	`char c = 'A';`
`boolean`	1 bit	true or false	false	`boolean flag = true;`

> **Best Practice**
>
> Use the smallest data type that can accommodate your data to save memory. For most purposes, use `int` for integers and `double` for decimals. Always add `L` suffix for long values and `f` suffix for float values.

Reference Data Types

Reference types store the memory address (reference) of an object rather than the actual value:

```
// Strings (reference type)
String firstName = "John";
String lastName = "Doe";
String fullName = firstName + " " + lastName;

// Arrays (reference type)
int[] numbers = new int[5];     // Array of 5 integers
String[] fruits = {"Apple", "Banana", "Cherry"};

// Objects (reference type)
Scanner scanner = new Scanner(System.in);
Date today = new Date();

// All classes, interfaces, enums, and arrays are reference types
// Default value for reference types is null
```

Variable Naming Rules

Table 2-2: Java Naming Conventions

Rule	Valid	Invalid
Can contain letters, digits, underscore, $	`myVar`, `my_var`, `$var`, `var1`	`my-var`, `my var`
Must start with letter, _, or $	`_count`, `$amount`, `value`	`1stName`, `123var`
Case-sensitive	`name` and `Name` are different	--
Cannot be a reserved keyword	--	`class`, `int`, `public`
Use camelCase for variables/methods	`studentName`, `calculateTotal`	`Student_Name`

Constants with final

```
// Constants are declared with 'final' keyword
// Convention: use UPPER_CASE with underscores

public class Constants {
    // Class-level constant
    public static final double PI = 3.14159;
    public static final int MAX_SIZE = 100;
    public static final String APP_NAME = "MyApplication";

    public static void main(String[] args) {
        // Local constant
        final int HOURS_IN_DAY = 24;
        // HOURS_IN_DAY = 25;  // ERROR: cannot reassign final variable

        System.out.println("PI = " + PI);
        System.out.println("Hours in a day: " + HOURS_IN_DAY);
    }
}
```

Scope of Variables

```java
public class VariableScope {
    // Instance variable (belongs to object)
    int instanceVar = 10;

    // Static variable (belongs to class)
    static int staticVar = 20;

    public void method() {
        // Local variable (method scope)
        int localVar = 30;

        if (true) {
            // Block scope
            int blockVar = 40;
            System.out.println(blockVar);  // Accessible here
        }
        // System.out.println(blockVar);  // ERROR: not accessible here

        System.out.println(localVar);      // Accessible
        System.out.println(instanceVar);   // Accessible
        System.out.println(staticVar);     // Accessible
    }
}
```

Variable Scope Rules

- **Local variables**: Declared inside a method; accessible only within that method
- **Instance variables**: Declared in class but outside methods; accessible by all methods of the class
- **Static variables**: Shared across all instances of the class
- **Block variables**: Declared inside a block (if, for, while); accessible only within that block

2.2 Operators

> **Definition: Operator**
>
> An **operator** is a special symbol that performs a specific operation on one, two, or three operands and returns a result.

Arithmetic Operators

```
public class ArithmeticOperators {
    public static void main(String[] args) {
        int a = 17;
        int b = 5;

        System.out.println("a + b = " + (a + b));   // 22 (Addition)
        System.out.println("a - b = " + (a - b));   // 12 (Subtraction)
              System.out.println("a * b = " + (a * b));     // 85
(Multiplication)
          System.out.println("a / b = " + (a / b));    // 3  (Integer
division)
               System.out.println("a % b = " + (a % b));     // 2
(Modulo/Remainder)

        // Integer division truncates decimal
         double result = (double) a / b;  // Cast to double for decimal
result
        System.out.println("a / b (double) = " + result);  // 3.4
    }
}
```

Assignment Operators

```
int x = 10;

x += 5;   // x = x + 5;  → x = 15
x -= 3;   // x = x - 3;  → x = 12
x *= 2;   // x = x * 2;  → x = 24
x /= 4;   // x = x / 4;  → x = 6
x %= 4;   // x = x % 4;  → x = 2

// Increment and Decrement
int y = 5;
int preIncrement = ++y;   // y becomes 6, then preIncrement = 6
int postIncrement = y++;  // postIncrement = 6, then y becomes 7

int z = 5;
int preDecrement = --z;   // z becomes 4, then preDecrement = 4
int postDecrement = z--;  // postDecrement = 4, then z becomes 3
```

Comparison (Relational) Operators

Table 2-3: Java Comparison Operators

Operator	Name	Example	Result (a=10, b=5)
`==`	Equal to	`a == b`	false
`!=`	Not equal to	`a != b`	true
`>`	Greater than	`a > b`	true
`<`	Less than	`a < b`	false
`>=`	Greater than or equal	`a >= b`	true
`<=`	Less than or equal	`a <= b`	false

Logical Operators

```
public class LogicalOperators {
    public static void main(String[] args) {
        boolean a = true;
        boolean b = false;

        // AND: both must be true
        System.out.println("a && b = " + (a && b));  // false

        // OR: at least one must be true
        System.out.println("a || b = " + (a || b));  // true

        // NOT: inverts the boolean value
        System.out.println("!a = " + (!a));           // false
        System.out.println("!b = " + (!b));           // true

        // Short-circuit evaluation
        int x = 5;
        // The second condition is not evaluated if the first is false
          if (x > 10 && (x / 0 == 0)) {   // No error, second part not
evaluated
            System.out.println("This won't print");
        }
    }
}
```

Bitwise Operators

```
public class BitwiseOperators {
    public static void main(String[] args) {
        int a = 5;   // Binary: 0101
        int b = 3;   // Binary: 0011

        System.out.println("a & b  = " + (a & b));   // 1  (0001) - AND
        System.out.println("a | b  = " + (a | b));   // 7  (0111) - OR
        System.out.println("a ^ b  = " + (a ^ b));   // 6  (0110) - XOR
         System.out.println("~a      = " + (~a));       // -6 (inverted) -
NOT
        System.out.println("a << 1 = " + (a << 1));  // 10 (1010) - Left
shift
          System.out.println("a >> 1 = " + (a >> 1));   // 2  (0010) -
Right shift

        // Practical: checking if a number is even/odd using bitwise AND
        int num = 7;
        if ((num & 1) == 0) {
            System.out.println(num + " is even");
        } else {
            System.out.println(num + " is odd");
        }
    }
}
```

Ternary Operator

```
public class TernaryOperator {
    public static void main(String[] args) {
        // Syntax: condition ? valueIfTrue : valueIfFalse

        int age = 20;
        String status = (age ⩾ 18) ? "Adult" : "Minor";
        System.out.println("Status: " + status);  // Adult

        int a = 10, b = 20;
        int max = (a > b) ? a : b;
        System.out.println("Maximum: " + max);  // 20

        // Nested ternary (use sparingly for readability)
        int score = 85;
        char grade = (score ⩾ 90) ? 'A' : (score ⩾ 80) ? 'B' : 'C';
        System.out.println("Grade: " + grade);  // B
    }
}
```

Operator Precedence

Table 2-4: Java Operator Precedence (Highest to Lowest)

Precedence	Operators	Associativity
1 (Highest)	`()` `[]` `.`	Left to right
2	`++` `--` (postfix), `++` `--` (prefix), `+` `-` (unary), `!` `~`	Right to left
3	`*` `/` `%`	Left to right
4	`+` `-`	Left to right
5	`<<` `>>` `>>>`	Left to right
6	`<` `⩽` `>` `⩾` `instanceof`	Left to right
7	`==` `≠`	Left to right
8	`&`	Left to right
9	`^`	Left to right
10	`|`	Left to right
11	`&&`	Left to right
12	`||`	Left to right
13	`? :` (ternary)	Right to left
14 (Lowest)	`=` `+=` `-=` etc.	Right to left

2.3 Type Casting

> **Definition: Type Casting**
>
> **Type casting** is the process of converting a value from one data type to another. Java supports two types: **implicit (widening)** casting and **explicit (narrowing)** casting.

Implicit Casting (Widening)

Implicit casting happens automatically when converting from a smaller type to a larger type:

```
// byte → short → int → long → float → double
// char → int → long → float → double

public class ImplicitCasting {
    public static void main(String[] args) {
        byte smallNum = 100;
        int bigNum = smallNum;      // byte to int (automatic)

        int integer = 100;
        long longNum = integer;     // int to long (automatic)

        long wholeNum = 100L;
        float decimal = wholeNum;   // long to float (automatic)

        float floatVal = 3.14f;
        double doubleVal = floatVal; // float to double (automatic)

        // char to int (ASCII value)
        char letter = 'A';
        int ascii = letter;         // 65

        System.out.println("ASCII of 'A': " + ascii);
    }
}
```

Explicit Casting (Narrowing)

```
// Explicit casting requires manual conversion using (type) syntax

public class ExplicitCasting {
    public static void main(String[] args) {
        double pi = 3.14159;
        int whole = (int) pi;       // Truncates decimal: 3

        long bigNum = 1000L;
        int smaller = (int) bigNum;  // May lose data if too large

        int large = 130;
        byte tiny = (byte) large;    // Overflow: -126 (wraps around)

        // Casting in expressions
        int a = 5, b = 2;
        double result = (double) a / b;  // 2.5 (not 2)

        System.out.println("whole = " + whole);
        System.out.println("result = " + result);
    }
}
```

> **Warning: Data Loss**
>
> Explicit casting from larger to smaller types can result in **data loss** or **overflow**. Always ensure the value fits within the target type's range before casting.

2.4 User Input

Java provides the `Scanner` class in the `java.util` package for reading user input from the console.

```java
import java.util.Scanner;

public class UserInput {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Reading a string
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();

        // Reading an integer
        System.out.print("Enter your age: ");
        int age = scanner.nextInt();

        // Reading a double
        System.out.print("Enter your height (in meters): ");
        double height = scanner.nextDouble();

        // Reading a boolean
        System.out.print("Are you a student (true/false): ");
        boolean isStudent = scanner.nextBoolean();

        System.out.println("\n--- User Information ---");
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Height: " + height);
        System.out.println("Student: " + isStudent);

        scanner.close();  // Always close the scanner when done
    }
}
```

Sample Output:

```
Enter your name: Alice
Enter your age: 25
Enter your height (in meters): 1.65
Are you a student (true/false): true

--- User Information ---
Name: Alice
Age: 25
Height: 1.65
Student: true
```

Scanner Methods Summary

Table 2-5: Common Scanner Methods

Method	Description	Return Type
`nextLine()`	Reads a line of text	String
`next()`	Reads a single token (word)	String
`nextInt()`	Reads an integer	int
`nextDouble()`	Reads a double	double
`nextBoolean()`	Reads a boolean	boolean
`nextLong()`	Reads a long	long
`next().charAt(0)`	Reads a single character	char
`hasNextInt()`	Checks if next token is an int	boolean

Important: Scanner Pitfall

When using `nextInt()` or `nextDouble()` followed by `nextLine()`, the newline character remains in the buffer. Call an extra `scanner.nextLine()` after reading a number to consume the leftover newline.

Chapter 3

Control Flow Statements

Control flow statements determine the order in which statements are executed in a program. They enable decision-making, looping, and branching.

3.1 If-Else Statements

> **Definition**
>
> The **if-else statement** is a conditional control structure that executes different blocks of code based on whether a boolean expression evaluates to true or false.

Simple If Statement

```
public class IfStatement {
    public static void main(String[] args) {
        int age = 20;

        if (age ⩾ 18) {
            System.out.println("You are an adult.");
        }

        // Single-line if (no braces needed, but recommended)
        if (age > 13)
            System.out.println("You are a teenager or older.");
    }
}
```

If-Else Statement

```
public class IfElse {
    public static void main(String[] args) {
        int number = -5;

        if (number ⩾ 0) {
            System.out.println(number + " is positive or zero.");
        } else {
            System.out.println(number + " is negative.");
        }
    }
}
```

Output:

```
-5 is negative.
```

If-Else-If Ladder

```
public class GradeEvaluator {
    public static void main(String[] args) {
        int score = 85;
        char grade;

        if (score ⩾ 90) {
            grade = 'A';
        } else if (score ⩾ 80) {
            grade = 'B';
        } else if (score ⩾ 70) {
            grade = 'C';
        } else if (score ⩾ 60) {
            grade = 'D';
        } else {
            grade = 'F';
        }

        System.out.println("Score: " + score + ", Grade: " + grade);
    }
}
```

Output:

```
Score: 85, Grade: B
```

Nested If Statements

```
public class NestedIf {
    public static void main(String[] args) {
        int age = 25;
        boolean hasLicense = true;

        if (age >= 18) {
            if (hasLicense) {
                System.out.println("You can drive.");
            } else {
                System.out.println("You need a license to drive.");
            }
        } else {
            System.out.println("You are too young to drive.");
        }
    }
}
```

3.2 Switch Statement

Definition

The **switch statement** evaluates an expression and matches its value against multiple `case` labels, executing the matching block of code. It provides a cleaner alternative to long if-else-if ladders.

Switch with Integers

```
public class SwitchExample {
    public static void main(String[] args) {
        int day = 3;
        String dayName;

        switch (day) {
            case 1:
                dayName = "Monday";
                break;
            case 2:
                dayName = "Tuesday";
                break;
            case 3:
                dayName = "Wednesday";
                break;
            case 4:
                dayName = "Thursday";
                break;
            case 5:
                dayName = "Friday";
                break;
            case 6:
                dayName = "Saturday";
                break;
            case 7:
                dayName = "Sunday";
                break;
            default:
                dayName = "Invalid day";
        }

        System.out.println("Day " + day + " is " + dayName);
    }
}
```

Output:

```
Day 3 is Wednesday
```

Switch with Strings (Java 7+)

```
public class SwitchString {
    public static void main(String[] args) {
        String fruit = "Apple";

        switch (fruit) {
            case "Apple":
                    System.out.println("An apple a day keeps the doctor
away.");
                break;
            case "Banana":
                System.out.println("Bananas are rich in potassium.");
                break;
            case "Orange":
                System.out.println("Oranges are full of Vitamin C.");
                break;
            default:
                System.out.println("Unknown fruit.");
        }
    }
}
```

Fall-Through Behavior

```
public class SwitchFallThrough {
    public static void main(String[] args) {
        String season = "Summer";

        switch (season) {
            case "Spring":
            case "Summer":
                System.out.println("It's warm outside!");
                break;  // Both Spring and Summer execute this
            case "Fall":
            case "Autumn":
            case "Winter":
                System.out.println("It's cold outside!");
                break;
            default:
                System.out.println("Unknown season.");
        }
    }
}
```

> **Warning: Missing Break**
>
> Without a `break` statement, execution "falls through" to the next case. This can cause unintended behavior if not used intentionally.

Enhanced Switch Expression (Java 14+)

```
public class SwitchExpression {
    public static void main(String[] args) {
        int day = 5;

        // Arrow syntax with yield (no break needed)
        String type = switch (day) {
            case 1, 2, 3, 4, 5 -> {
                System.out.println("Working day");
                yield "Weekday";  // yield returns a value
            }
            case 6, 7 -> "Weekend";
            default -> "Invalid";
        };

        System.out.println("Day type: " + type);
    }
}
```

3.3 Loops (for, while, do-while)

> **Definition**
>
> A **loop** is a control structure that repeatedly executes a block of code until a specified condition is met. Java provides three loop constructs: `for`, `while`, and `do-while`.

The for Loop

```
// Syntax: for (initialization; condition; update) { body }

public class ForLoop {
    public static void main(String[] args) {
        // Print numbers 1 to 5
        for (int i = 1; i ⩽ 5; i++) {
            System.out.println("Iteration: " + i);
        }

        // Countdown
        for (int i = 5; i ⩾ 1; i--) {
            System.out.print(i + " ");
        }
        System.out.println("Blast off!");

        // Multiple variables in for loop
        for (int i = 1, j = 10; i ⩽ 10; i++, j--) {
            System.out.println("i=" + i + ", j=" + j);
        }
    }
}
```

The while Loop

```
public class WhileLoop {
    public static void main(String[] args) {
        // Syntax: while (condition) { body }

        int count = 1;
        while (count ⩽ 5) {
            System.out.println("Count: " + count);
            count++;
        }

        // User input validation example
        java.util.Scanner scanner = new java.util.Scanner(System.in);
        int number = -1;

        while (number < 0) {
            System.out.print("Enter a positive number: ");
            number = scanner.nextInt();
            if (number < 0) {
                System.out.println("Invalid! Try again.");
            }
        }
        System.out.println("You entered: " + number);
        scanner.close();
    }
}
```

The do-while Loop

```java
public class DoWhileLoop {
    public static void main(String[] args) {
        // Syntax: do { body } while (condition);
        // The body executes at least once

        int num = 1;
        do {
            System.out.println("Number: " + num);
            num++;
        } while (num <= 5);

        // Menu-driven program example
        java.util.Scanner scanner = new java.util.Scanner(System.in);
        int choice;

        do {
            System.out.println("\n--- Menu ---");
            System.out.println("1. Print Hello");
            System.out.println("2. Print World");
            System.out.println("3. Exit");
            System.out.print("Enter choice: ");
            choice = scanner.nextInt();

            switch (choice) {
                case 1 -> System.out.println("Hello!");
                case 2 -> System.out.println("World!");
                case 3 -> System.out.println("Goodbye!");
                default -> System.out.println("Invalid choice!");
            }
        } while (choice != 3);

        scanner.close();
    }
}
```

Enhanced for Loop (for-each)

```
public class EnhancedForLoop {
    public static void main(String[] args) {
        String[] fruits = {"Apple", "Banana", "Cherry", "Date"};

        // for (elementType variable : collection/array)
        for (String fruit : fruits) {
            System.out.println(fruit);
        }

        // With an ArrayList
           java.util.List<Integer> numbers = java.util.Arrays.asList(10,
20, 30, 40);

        for (int num : numbers) {
            System.out.println("Number: " + num);
        }

        // Cannot modify the collection during iteration
        // Cannot access the index directly
    }
}
```

Loop Comparison

Table 3-1: Java Loop Types Comparison

Feature	for	while	do-while	for-each
Best for	Known iterations	Unknown iterations	At least once	Collections/arrays
Condition check	Before iteration	Before iteration	After iteration	Automatic
May not execute	Yes	Yes	No	Yes (empty collection)
Index access	Built-in	Manual	Manual	Not directly
Modify collection	Possible	Possible	Possible	Not recommended

3.4 Break and Continue

> **Definition**
>
> **break** terminates the loop or switch statement entirely and transfers control to the statement following the loop. **continue** skips the current iteration and moves to the next iteration of the loop.

Break Statement

```
public class BreakExample {
    public static void main(String[] args) {
        // Break out of a loop early
        for (int i = 1; i ⩽ 10; i++) {
            if (i == 5) {
                break;  // Exit the loop when i is 5
            }
            System.out.print(i + " ");  // Prints: 1 2 3 4
        }
        System.out.println("\nLoop ended.");

        // Searching in an array
        int[] numbers = {10, 25, 30, 45, 50};
        int target = 30;
        int index = -1;

        for (int i = 0; i < numbers.length; i++) {
            if (numbers[i] == target) {
                index = i;
                break;  // Found it, no need to continue
            }
        }
        System.out.println("Found at index: " + index);
    }
}
```

Labeled Break

```
public class LabeledBreak {
    public static void main(String[] args) {
            // Labeled break exits the specified loop, not just the
innermost
        outer:  // Label for the outer loop
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 3; j++) {
                if (i == 2 && j == 2) {
                    break outer;  // Break out of the outer loop
                }
                System.out.println("i=" + i + ", j=" + j);
            }
        }
        System.out.println("Exited outer loop.");
    }
}
```

Continue Statement

```
public class ContinueExample {
    public static void main(String[] args) {
        // Skip even numbers
        for (int i = 1; i <= 10; i++) {
            if (i % 2 == 0) {
                continue;  // Skip the rest of this iteration
            }
            System.out.print(i + " ");  // Prints: 1 3 5 7 9
        }
        System.out.println();

        // Skip negative numbers in an array
        int[] values = {5, -3, 8, -1, 10, -7, 15};
        int sum = 0;

        for (int val : values) {
            if (val < 0) {
                continue;  // Skip negative values
            }
            sum += val;
        }
        System.out.println("Sum of positive numbers: " + sum);  // 38
    }
}
```

Labeled Continue

```
public class LabeledContinue {
    public static void main(String[] args) {
          // Labeled continue skips to next iteration of the specified
loop
        outer:
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 3; j++) {
                if (j == 2) {
                            continue outer;   // Skip to next outer loop
iteration
                }
                System.out.println("i=" + i + ", j=" + j);
            }
        }
    }
}
```

Best Practice

Use `break` and `continue` judiciously. Overuse can make code harder to read. Consider refactoring complex nested loops with multiple breaks into separate methods for better readability.

Chapter 4

Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm that organizes code around objects rather than functions. Java is fundamentally an object-oriented language, and understanding OOP principles is essential for mastering Java.

4.1 Classes and Objects

> **Definition: Class**
>
> A **class** is a blueprint or template that defines the properties (fields/attributes) and behaviors (methods) that objects of that type will have. It is a logical construct that does not occupy memory until an object is created.

> **Definition: Object**
>
> An **object** is an instance of a class. It is a concrete entity created from a class blueprint that occupies memory and has actual values for its attributes. Multiple objects can be created from a single class.

```
// Class definition
public class Car {
    // Fields (instance variables) - describe the object's state
    String brand;
    String model;
    int year;
    String color;
    double speed;

    // Methods - describe the object's behavior
    void accelerate(double amount) {
        speed += amount;
            System.out.println(brand + " accelerated to " + speed + "
km/h");
    }

    void brake(double amount) {
        speed = Math.max(0, speed - amount);
        System.out.println(brand + " slowed to " + speed + " km/h");
    }

    void displayInfo() {
        System.out.println("Brand: " + brand);
        System.out.println("Model: " + model);
        System.out.println("Year: " + year);
        System.out.println("Color: " + color);
        System.out.println("Speed: " + speed + " km/h");
    }
}
```

```
// Creating and using objects
public class CarDemo {
    public static void main(String[] args) {
        // Create objects using the 'new' keyword
        Car car1 = new Car();
        car1.brand = "Toyota";
        car1.model = "Corolla";
        car1.year = 2024;
        car1.color = "Blue";
        car1.speed = 0;

        Car car2 = new Car();
        car2.brand = "Honda";
        car2.model = "Civic";
        car2.year = 2023;
        car2.color = "Red";
        car2.speed = 0;

        // Call methods on objects
        car1.accelerate(50);
        car1.accelerate(30);
        car1.brake(20);

        System.out.println();
        car1.displayInfo();

        System.out.println();
        car2.displayInfo();
    }
}
```

Output:

```
Toyota accelerated to 50.0 km/h
Toyota accelerated to 80.0 km/h
Toyota slowed to 60.0 km/h

Brand: Toyota
Model: Corolla
Year: 2024
Color: Blue
Speed: 60.0 km/h

Brand: Honda
Model: Civic
Year: 2023
Color: Red
Speed: 0.0 km/h
```

Key Concepts

- **new** keyword allocates memory and creates an instance
- **dot operator (.)** accesses fields and methods of an object
- Each object has its own copy of instance variables
- Objects are reference types — variables hold references (memory addresses)

4.2 Constructors

> **Definition: Constructor**
>
> A **constructor** is a special method that is automatically invoked when an object is created. It has the same name as the class, no return type (not even void), and is used to initialize the object's state.

Default Constructor

```java
public class Student {
    String name;
    int age;

    // Default constructor (no parameters)
    public Student() {
        name = "Unknown";
        age = 0;
    }

    public static void main(String[] args) {
        Student s = new Student();  // Calls default constructor
        System.out.println(s.name + ", " + s.age);  // Unknown, 0
    }
}
```

> **Important**
>
> If you do not define any constructor, Java automatically provides a default no-arg constructor. However, if you define any constructor, the default constructor is **not** provided automatically.

Parameterized Constructor

```
public class Student {
    String name;
    int age;
    double gpa;

    // Parameterized constructor
    public Student(String name, int age, double gpa) {
        this.name = name;  // this refers to the current object
        this.age = age;
        this.gpa = gpa;
    }

    // Multiple constructors (constructor overloading)
    public Student(String name) {
        this(name, 0, 0.0);  // Call another constructor
    }

    void display() {
        System.out.println(name + " | Age: " + age + " | GPA: " + gpa);
    }

    public static void main(String[] args) {
        Student s1 = new Student("Alice", 20, 3.8);
        Student s2 = new Student("Bob");

        s1.display();  // Alice | Age: 20 | GPA: 3.8
        s2.display();  // Bob | Age: 0 | GPA: 0.0
    }
}
```

Constructor Chaining

```
public class Employee {
    String name;
    int id;
    String department;
    double salary;

    // Full constructor
      public Employee(String name, int id, String department, double
salary) {
        this.name = name;
        this.id = id;
        this.department = department;
        this.salary = salary;
    }

    // Chain to full constructor
    public Employee(String name, int id) {
        this(name, id, "Unassigned", 0.0);
    }

    // Chain to two-arg constructor
    public Employee(String name) {
        this(name, 0);
    }

    // Chain to single-arg constructor
    public Employee() {
        this("Unknown");
    }
}
```

4.3 Encapsulation

> **Definition: Encapsulation**
>
> **Encapsulation** is the OOP principle of bundling data (fields) and the methods that operate on that data within a single unit (class), and restricting direct access to some of the object's components. It is achieved through access modifiers and getter/setter methods.

```
public class BankAccount {
    // Private fields — cannot be accessed directly from outside
    private String accountNumber;
    private String accountHolder;
    private double balance;
    private static int accountCounter = 1000;

    // Constructor
    public BankAccount(String accountHolder, double initialDeposit) {
        this.accountHolder = accountHolder;
        this.balance = initialDeposit;
        this.accountNumber = "ACC" + (++accountCounter);
    }

    // Public getter methods — controlled read access
    public String getAccountNumber() {
        return accountNumber;
    }

    public String getAccountHolder() {
        return accountHolder;
    }

    public double getBalance() {
        return balance;
    }

    // Setter with validation
    public void setAccountHolder(String accountHolder) {
        if (accountHolder ≠ null && !accountHolder.isEmpty()) {
            this.accountHolder = accountHolder;
        }
    }

    // Business methods — controlled write access
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: $" + amount);
        } else {
            System.out.println("Invalid deposit amount!");
        }
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount ⩽ balance) {
```

```
public class BankDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount("John Doe", 500.0);

        account.displayInfo();

        // account.balance = 1000000;  // ERROR: balance is private
        account.deposit(250.0);
        account.withdraw(100.0);

                        System.out.println("\nCurrent   Balance:   $"   +
account.getBalance());
    }
}
```

Access Modifiers

Table 4-1: Java Access Modifiers

Modifier	Same Class	Same Package	Subclass	Anywhere
`public`	Yes	Yes	Yes	Yes
`protected`	Yes	Yes	Yes	No
`default` (no modifier)	Yes	Yes	No	No
`private`	Yes	No	No	No

4.4 Inheritance

> **Definition: Inheritance**
>
> **Inheritance** is an OOP mechanism where a new class (subclass/child) derives properties and methods from an existing class (superclass/parent). It promotes code reusability and establishes an "is-a" relationship between classes.

```
// Parent class (Superclass)
public class Animal {
    protected String name;
    protected int age;

    public Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void eat() {
        System.out.println(name + " is eating.");
    }

    public void sleep() {
        System.out.println(name + " is sleeping.");
    }

    public void makeSound() {
        System.out.println(name + " makes a sound.");
    }

    public void displayInfo() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}
```

```
// Child class (Subclass)
public class Dog extends Animal {
    private String breed;

    public Dog(String name, int age, String breed) {
        super(name, age);  // Call parent constructor
        this.breed = breed;
    }

    // Override parent method
    @Override
    public void makeSound() {
        System.out.println(name + " barks: Woof! Woof!");
    }

    // Additional method specific to Dog
    public void fetch() {
        System.out.println(name + " is fetching the ball.");
    }

    public void displayBreed() {
        System.out.println(name + " is a " + breed);
    }
}
```

```
public class Cat extends Animal {
    private boolean isIndoor;

    public Cat(String name, int age, boolean isIndoor) {
        super(name, age);
        this.isIndoor = isIndoor;
    }

    @Override
    public void makeSound() {
        System.out.println(name + " meows: Meow!");
    }

    public void climb() {
        System.out.println(name + " is climbing.");
    }
}
```

```
public class InheritanceDemo {
    public static void main(String[] args) {
        Dog dog = new Dog("Buddy", 3, "Golden Retriever");
        Cat cat = new Cat("Whiskers", 2, true);

        System.out.println("== Dog ==");
        dog.displayInfo();
        dog.eat();           // Inherited from Animal
        dog.makeSound();     // Overridden in Dog
        dog.fetch();         // Specific to Dog

        System.out.println("\n== Cat ==");
        cat.displayInfo();
        cat.eat();           // Inherited from Animal
        cat.makeSound();     // Overridden in Cat
        cat.climb();         // Specific to Cat
    }
}
```

Output:

```
== Dog ==
Name: Buddy, Age: 3
Buddy is eating.
Buddy barks: Woof! Woof!
Buddy is fetching the ball.

== Cat ==
Name: Whiskers, Age: 2
Whiskers is eating.
Whiskers meows: Meow!
Whiskers is climbing.
```

The super Keyword

```
public class ElectricCar extends Car {
    private int batteryCapacity;

        public  ElectricCar(String  brand,  String  model,  int  year,  int
battery) {
        super(brand, model, year);  // Call parent constructor
        this.batteryCapacity = battery;
    }

    @Override
    public void displayInfo() {
        super.displayInfo();  // Call parent's displayInfo
        System.out.println("Battery: " + batteryCapacity + " kWh");
    }
}
```

Inheritance Best Practices

- Use inheritance only when there is a true "is-a" relationship
- Java supports **single inheritance** only (one direct parent)
- Use `final` keyword to prevent a class from being inherited
- Use `final` on methods to prevent overriding
- Always annotate overridden methods with `@Override`

4.5 Polymorphism

Definition: Polymorphism

Polymorphism (Greek for "many forms") is the ability of an object to take on many forms. In Java, it allows a subclass object to be treated as an instance of its parent class, and enables a single method to behave differently based on the object that invokes it.

Compile-Time Polymorphism (Method Overloading)

```
public class Calculator {
    // Same method name, different parameters

    public int add(int a, int b) {
        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }

    public String add(String a, String b) {
        return a + b;  // Concatenation
    }

    public static void main(String[] args) {
        Calculator calc = new Calculator();

        System.out.println(calc.add(5, 3));          // 8 (int)
        System.out.println(calc.add(5.5, 3.3));      // 8.8 (double)
        System.out.println(calc.add(1, 2, 3));       // 6 (3 params)
           System.out.println(calc.add("Hello, ", "World"));   // Hello,
World
    }
}
```

Runtime Polymorphism (Method Overriding + Dynamic Dispatch)

```
// Runtime polymorphism example
public class PolymorphismDemo {
    public static void main(String[] args) {
        // Parent reference can hold child objects
        Animal animal1 = new Dog("Buddy", 3, "Labrador");
        Animal animal2 = new Cat("Mittens", 2, false);
        Animal animal3 = new Animal("Generic", 5);

        // Same method call, different behavior at runtime
        animal1.makeSound();  // Buddy barks
        animal2.makeSound();  // Mittens meows
        animal3.makeSound();  // Generic makes a sound

        // Array of parent type holding different child objects
        Animal[] animals = {animal1, animal2, animal3};

        System.out.println("\n--- Animal Sounds ---");
        for (Animal animal : animals) {
            animal.makeSound();  // Polymorphic method call
        }
    }
}
```

instanceof Operator

```
public class InstanceofDemo {
    public static void main(String[] args) {
        Animal animal = new Dog("Buddy", 3, "Labrador");

        // Check the actual type of object
        if (animal instanceof Dog) {
            System.out.println("This is a Dog!");
            Dog dog = (Dog) animal;  // Downcasting
            dog.fetch();
        }

        if (animal instanceof Animal) {
            System.out.println("This is an Animal!");
        }

        // Java 16+ pattern matching for instanceof
        if (animal instanceof Dog dog) {
            dog.fetch();  // No explicit cast needed
        }
    }
}
```

4.6 Abstraction

> **Definition: Abstraction**
>
> **Abstraction** is the OOP principle of hiding complex implementation details and showing only the essential features of an object. It is achieved in Java through **abstract classes** and **interfaces**.

Abstract Class

```
// Abstract class - cannot be instantiated directly
public abstract class Shape {
    protected String color;

    public Shape(String color) {
        this.color = color;
    }

    // Abstract method - no implementation, must be overridden
    public abstract double calculateArea();

    // Concrete method - can be used by subclasses
    public void displayColor() {
        System.out.println("Color: " + color);
    }

    // Concrete method that uses abstract method
    public void printArea() {
        System.out.println("Area: " + calculateArea());
    }
}
```

```
public class Circle extends Shape {
    private double radius;

    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        return Math.PI * radius * radius;
    }

    public static void main(String[] args) {
        Circle circle = new Circle("Red", 5.0);
        circle.displayColor();
        circle.printArea();
    }
}
```

Output:

```
Color: Red
Area: 78.53981633974483
```

```
public class Rectangle extends Shape {
    private double width;
    private double height;

    public Rectangle(String color, double width, double height) {
        super(color);
        this.width = width;
        this.height = height;
    }

    @Override
    public double calculateArea() {
        return width * height;
    }
}
```

Abstract Class Rules

- Cannot instantiate an abstract class directly
- Can contain both abstract and concrete methods
- A class extending an abstract class must implement all abstract methods
- Can have constructors (called by subclass constructors via super())
- Can have fields, static methods, and final methods

4.7 Interfaces

> **Definition: Interface**
>
> An **interface** is a reference type in Java that defines a contract of abstract methods (and since Java 8, default and static methods) that implementing classes must provide. It represents a "can-do" relationship and enables a form of multiple inheritance.

```
// Interface definition
public interface Drawable {
    // Implicitly public, static, and final
    String TOOL = "Pencil";

    // Abstract method (implicitly public and abstract)
    void draw();

    // Default method (Java 8+) — has implementation
    default void printInfo() {
        System.out.println("This object can be drawn.");
    }

    // Static method (Java 8+)
    static void describeTool() {
        System.out.println("Drawing tool: " + TOOL);
    }
}
```

```
public interface Resizable {
    void resize(double scale);
    void resetSize();
}
```

```
// A class can implement multiple interfaces
public class Rectangle implements Drawable, Resizable {
    private double width;
    private double height;
    private String color;

    public Rectangle(double width, double height, String color) {
        this.width = width;
        this.height = height;
        this.color = color;
    }

    @Override
    public void draw() {
        System.out.println("Drawing a " + color + " rectangle ("
            + width + "x" + height + ")");
    }

    @Override
    public void resize(double scale) {
        width *= scale;
        height *= scale;
    }

    @Override
    public void resetSize() {
        width = 0;
        height = 0;
    }

    public static void main(String[] args) {
        Rectangle rect = new Rectangle(10, 20, "blue");

        rect.draw();
        rect.printInfo();  // Default method from interface

        rect.resize(1.5);
        rect.draw();

        Drawable.describeTool();  // Static method from interface

        // Polymorphism with interfaces
        Drawable d = rect;
        d.draw();

        Resizable r = rect;
```

Functional Interfaces

```
// A functional interface has exactly one abstract method
@FunctionalInterface
public interface Greeting {
    void sayHello(String name);
    // Cannot have more than one abstract method
}

// Usage with lambda expression (covered in detail in Chapter 8)
public class FunctionalInterfaceDemo {
    public static void main(String[] args) {
           Greeting greeting = (name) -> System.out.println("Hello, " +
name + "!");
        greeting.sayHello("Alice");
    }
}
```

Abstract Class vs Interface

Table 4-2: Abstract Class vs Interface

Feature	Abstract Class	Interface
Keyword	`abstract class`	`interface`
Instantiation	Cannot be instantiated	Cannot be instantiated
Methods	Abstract and concrete	Abstract, default, static (Java 8+)
Multiple inheritance	Single (extends one class)	Multiple (implements many)
Variables	Any type	Implicitly public, static, final
Constructors	Yes	No
Access modifiers	Any	Methods implicitly public
Use case	"Is-a" relationship with shared code	"Can-do" capability contract

Chapter 5

Arrays and Strings

5.1 Arrays

> **Definition: Array**
>
> An **array** is a fixed-size, ordered collection of elements of the same data type stored in contiguous memory locations. Each element is accessed by an index, starting from 0.

Declaring and Creating Arrays

```
// Method 1: Declare and allocate
int[] numbers = new int[5];  // Array of 5 integers (default: 0)

// Method 2: Declare with initial values
int[] scores = {85, 90, 78, 92, 88};

// Method 3: Anonymous array
String[] fruits = new String[]{"Apple", "Banana", "Cherry"};

// Method 4: Create and fill later
double[] temperatures;
temperatures = new double[7];

// Accessing and modifying elements
numbers[0] = 10;
numbers[1] = 20;
System.out.println(numbers[0]);  // 10

// Array length (read-only property)
System.out.println("Array size: " + scores.length);  // 5
```

Iterating Over Arrays

```
public class ArrayIteration {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};

        // Using for loop with index
        System.out.println("Using for loop:");
        for (int i = 0; i < numbers.length; i++) {
            System.out.println("Index " + i + ": " + numbers[i]);
        }

        // Using enhanced for loop (for-each)
        System.out.println("\nUsing for-each:");
        for (int num : numbers) {
            System.out.println(num);
        }

        // Common operations
        // Sum all elements
        int sum = 0;
        for (int num : numbers) {
            sum += num;
        }
        System.out.println("\nSum: " + sum);

        // Find maximum
        int max = numbers[0];
        for (int num : numbers) {
            if (num > max) {
                max = num;
            }
        }
        System.out.println("Maximum: " + max);
    }
}
```

Arrays Class Utility Methods

```
import java.util.Arrays;

public class ArraysClassDemo {
    public static void main(String[] args) {
        int[] arr1 = {5, 2, 8, 1, 9, 3};
        int[] arr2 = {5, 2, 8, 1, 9, 3};

        // Sort array
        Arrays.sort(arr1);
        System.out.println("Sorted: " + Arrays.toString(arr1));
        // [1, 2, 3, 5, 8, 9]

        // Binary search (array must be sorted)
        int index = Arrays.binarySearch(arr1, 5);
        System.out.println("Index of 5: " + index);  // 3

        // Fill array with a value
        int[] filled = new int[5];
        Arrays.fill(filled, 7);
        System.out.println("Filled: " + Arrays.toString(filled));
        // [7, 7, 7, 7, 7]

        // Compare arrays
         System.out.println("Equals: " + Arrays.equals(arr1, arr2));  //
false

        // Copy array
        int[] copied = Arrays.copyOf(arr1, arr1.length);
        System.out.println("Copied: " + Arrays.toString(copied));

        // Copy range
        int[] partial = Arrays.copyOfRange(arr1, 1, 4);
        System.out.println("Partial: " + Arrays.toString(partial));
        // [2, 3, 5]
    }
}
```

5.2 Multi-Dimensional Arrays

> **Definition**
>
> A **multi-dimensional array** is an array of arrays. The most common is a two-dimensional array, which represents data in rows and columns like a table or matrix.

```
public class MultiDimensionalArrays {
    public static void main(String[] args) {
        // 2D array declaration and initialization
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        // Accessing elements
        System.out.println("Element at [1][2]: " + matrix[1][2]);  // 6

        // Iterate using nested loops
        System.out.println("\nMatrix:");
        for (int i = 0; i < matrix.length; i++) {  // Rows
            for (int j = 0; j < matrix[i].length; j++) {  // Columns
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
        }

        // Using enhanced for loop
        System.out.println("\nUsing enhanced for:");
        for (int[] row : matrix) {
            for (int element : row) {
                System.out.print(element + " ");
            }
            System.out.println();
        }

        // Jagged array (rows of different lengths)
        int[][] jagged = {
            {1, 2},
            {3, 4, 5},
            {6, 7, 8, 9}
        };

        System.out.println("\nJagged array:");
        for (int[] row : jagged) {
            System.out.println("Row length: " + row.length);
            for (int val : row) {
                System.out.print(val + " ");
            }
            System.out.println();
        }
```

5.3 String Class

> **Definition**
>
> The **String** class in Java represents an immutable sequence of characters. Once created, a String object cannot be modified. Any operation that appears to modify a String actually creates a new String object.

Creating Strings

```
public class StringCreation {
    public static void main(String[] args) {
        // Method 1: String literal (stored in String pool)
        String s1 = "Hello";
        String s2 = "Hello";  // Reuses the same object from pool
        System.out.println("s1 == s2: " + (s1 == s2));  // true

        // Method 2: Using new keyword (creates separate object in heap)
        String s3 = new String("Hello");
        System.out.println("s1 == s3: " + (s1 == s3));  // false
        System.out.println("s1.equals(s3): " + s1.equals(s3));  // true

        // Method 3: From character array
        char[] chars = {'H', 'e', 'l', 'l', 'o'};
        String s4 = new String(chars);

        // Method 4: From byte array
        byte[] bytes = {72, 101, 108, 108, 111};  // ASCII for "Hello"
        String s5 = new String(bytes);

        System.out.println(s4);  // Hello
        System.out.println(s5);  // Hello
    }
}
```

> **Important**
>
> Always use `.equals()` to compare String **content**, not `==`. The `==` operator compares references (memory addresses), not the actual character content. Use `==` only when checking if two references point to the exact same object.

Common String Methods

```
public class StringMethods {
    public static void main(String[] args) {
        String text = "  Hello, Java World!  ";

        // Length
        System.out.println("Length: " + text.length());

        // Trim whitespace
        String trimmed = text.trim();
        System.out.println("Trimmed: '" + trimmed + "'");

        // Case conversion
        System.out.println("Upper: " + trimmed.toUpperCase());
        System.out.println("Lower: " + trimmed.toLowerCase());

        // Substring
        System.out.println("Substring(7): " + trimmed.substring(7));
            System.out.println("Substring(0,5):  " +  trimmed.substring(0,
5));

        // Character at index
        System.out.println("Char at 7: " + trimmed.charAt(7));

        // Index of character/substring
                        System.out.println("Index   of   'Java':   "   +
trimmed.indexOf("Java"));
        System.out.println("Index of 'o': " + trimmed.indexOf('o'));
                       System.out.println("Last   index   of   'o':   "   +
trimmed.lastIndexOf('o'));

        // Contains
                         System.out.println("Contains   'Java':   "   +
trimmed.contains("Java"));

        // Starts with / Ends with
                       System.out.println("Starts   with   'Hello':   "   +
trimmed.startsWith("Hello"));
        System.out.println("Ends with '!': " + trimmed.endsWith("!"));

        // Replace
              System.out.println("Replace:  "  +  trimmed.replace("Java",
"Python"));

        // Split
        String[] words = trimmed.split(" ");
        System.out.println("Words: " + String.join(" | ", words));
```

String Immutability

```
public class StringImmutability {
    public static void main(String[] args) {
        String s = "Hello";

        // concat() returns a NEW String, doesn't modify original
        String s2 = s.concat(" World");

        System.out.println("Original: " + s);     // Hello
        System.out.println("New: " + s2);         // Hello World

        // Similarly, toUpperCase() returns new String
        String upper = s.toUpperCase();
        System.out.println("Original: " + s);     // Hello
        System.out.println("Upper: " + upper);    // HELLO

        // This creates multiple intermediate String objects
        String result = "";
        for (int i = 0; i < 1000; i++) {
            result += i;  // Inefficient: creates new String each time
        }

        // Better approach: use StringBuilder
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 1000; i++) {
            sb.append(i);  // Efficient: modifies same object
        }
        String efficient = sb.toString();
    }
}
```

5.4 StringBuilder and StringBuffer

> **Definition**
>
> **StringBuilder** and **StringBuffer** are mutable sequence of characters. Unlike String, they can be modified without creating new objects. StringBuilder is faster but not thread-safe, while StringBuffer is thread-safe (synchronized) but slightly slower.

```
public class StringBuilderDemo {
    public static void main(String[] args) {
        // Creating StringBuilder
        StringBuilder sb = new StringBuilder("Hello");

        // Append
        sb.append(" World");
        System.out.println("After append: " + sb);

        // Insert
        sb.insert(5, ",");
        System.out.println("After insert: " + sb);

        // Replace
        sb.replace(7, 12, "Java");
        System.out.println("After replace: " + sb);

        // Delete
        sb.delete(5, 6);
        System.out.println("After delete: " + sb);

        // Reverse
        sb.reverse();
        System.out.println("After reverse: " + sb);
        sb.reverse();  // Back to normal

        // Character at index
        System.out.println("Char at 0: " + sb.charAt(0));

        // Length and capacity
        System.out.println("Length: " + sb.length());
        System.out.println("Capacity: " + sb.capacity());

        // Convert to String
        String result = sb.toString();
        System.out.println("String: " + result);

        // Method chaining
        StringBuilder chain = new StringBuilder();
        chain.append("Java")
             .append(" is ")
             .append("awesome!");
        System.out.println("Chained: " + chain);
    }
}
```

StringBuffer (Thread-Safe Version)

```
public class StringBufferDemo {
    public static void main(String[] args) {
        // StringBuffer has the same API as StringBuilder
        // but all methods are synchronized (thread-safe)

        StringBuffer buffer = new StringBuffer("Thread-Safe");

        buffer.append(" String");
        buffer.insert(0, "[Start] ");

        System.out.println(buffer);  // [Start] Thread-Safe String

        // Use StringBuffer when multiple threads modify the same string
        // Use StringBuilder in single-threaded contexts (faster)
    }
}
```

String vs StringBuilder vs StringBuffer

Table 5-1: String Comparison

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread-Safe	Yes (immutable)	No	Yes (synchronized)
Performance	Slow for modifications	Fast	Moderate
Use case	Fixed text	Single-threaded string building	Multi-threaded string building
Since	Java 1.0	Java 5	Java 1.0

Chapter 6

Exception Handling

Exception handling is a mechanism that allows a program to deal with runtime errors gracefully without crashing. It separates error-handling code from normal business logic.

6.1 Types of Exceptions

> **Definition: Exception**
>
> An **exception** is an event that disrupts the normal flow of program execution. It is an object that is thrown at the point where an error occurs and can be caught and handled by the program.

Exception Hierarchy

All exceptions in Java inherit from the `java.lang.Throwable` class. The hierarchy has two main branches:

Table 6-1: Java Exception Hierarchy

Category	Description	Handling	Examples
Error	Serious system-level problems	Generally not caught	OutOfMemoryError, StackOverflowError
Checked Exception	Compile-time checked exceptions	Must be caught or declared	IOException, SQLException, FileNotFoundException
Unchecked Exception	Runtime exceptions	Optional handling	NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException

Common Built-in Exceptions

```
public class CommonExceptions {
    public static void main(String[] args) {
        // ArithmeticException
        try {
            int result = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero!");
        }

        // NullPointerException
        try {
            String s = null;
            int len = s.length();
        } catch (NullPointerException e) {
            System.out.println("String is null!");
        }

        // ArrayIndexOutOfBoundsException
        try {
            int[] arr = new int[5];
            int val = arr[10];
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Index out of bounds!");
        }

        // NumberFormatException
        try {
            int num = Integer.parseInt("abc");
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format!");
        }

        // ClassCastException
        try {
            Object obj = "Hello";
            Integer num = (Integer) obj;
        } catch (ClassCastException e) {
            System.out.println("Invalid type cast!");
        }
    }
}
```

6.2 Try-Catch-Finally

> **Definition**
>
> The **try-catch-finally** block is the fundamental exception handling structure. Code that might throw an exception goes in the `try` block, handling code goes in `catch` blocks, and cleanup code goes in `finally`.

Basic Try-Catch

```
public class BasicTryCatch {
    public static void main(String[] args) {
        try {
            int[] numbers = {1, 2, 3};
            int result = numbers[5];  // This will throw exception
            System.out.println(result);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Index out of bounds!");
            System.out.println("Message: " + e.getMessage());
        }

        System.out.println("Program continues normally...");
    }
}
```

Output:

```
Error: Index out of bounds!
Message: Index 5 out of bounds for length 3
Program continues normally...
```

Multiple Catch Blocks

```
public class MultipleCatch {
    public static void main(String[] args) {
        try {
            String input = null;

            // This could throw different exceptions
            int num = Integer.parseInt(input);  // NullPointerException
            int result = 10 / num;

        } catch (NumberFormatException e) {
            System.out.println("Invalid number format!");
        } catch (ArithmeticException e) {
            System.out.println("Math error!");
        } catch (NullPointerException e) {
            System.out.println("Value is null!");
        } catch (Exception e) {
                            System.out.println("Some other error: " +
e.getClass().getName());
        }
    }
}
```

Important: Catch Order

When using multiple catch blocks, always order them from **most specific to most general**. Placing `catch (Exception e)` first would make all other catch blocks unreachable.

Finally Block

```
import java.io.*;

public class FinallyBlock {
    public static void main(String[] args) {
        BufferedReader reader = null;

        try {
            reader = new BufferedReader(new FileReader("test.txt"));
            String line = reader.readLine();
            System.out.println(line);
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        } finally {
            // Always executes, even if exception occurred
            System.out.println("Cleaning up resources...");
            try {
                if (reader ≠ null) {
                    reader.close();
                }
            } catch (IOException e) {
                System.out.println("Error closing file.");
            }
        }
    }
}
```

Best Practice

Use the **try-with-resources** statement (Java 7+) for AutoCloseable resources. It automatically closes resources at the end, eliminating the need for a finally block:

```
public class TryWithResources {
    public static void main(String[] args) {
        // Resources declared in try() are auto-closed
        try (BufferedReader reader = new BufferedReader(
                new FileReader("test.txt"))) {
            String line = reader.readLine();
            System.out.println(line);
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
        // reader is automatically closed here
    }
}
```

Multi-Catch Block (Java 7+)

```
public class MultiCatch {
    public static void main(String[] args) {
        try {
            // Some risky operation
            int[] arr = {1, 2, 3};
            System.out.println(arr[5]);
          } catch (ArrayIndexOutOfBoundsException | NullPointerException
e) {
            // Handle multiple exceptions with same handler
                        System.out.println("Array or null error: " +
e.getMessage());
        }
    }
}
```

6.3 Throw and Throws

throw Keyword

```
public class ThrowExample {
    public static void checkAge(int age) {
        if (age < 0) {
             throw new IllegalArgumentException("Age cannot be negative:
" + age);
        }
        if (age < 18) {
            throw new IllegalArgumentException("Must be 18 or older");
        }
        System.out.println("Age is valid: " + age);
    }

    public static void main(String[] args) {
        try {
            checkAge(25);   // OK
            checkAge(-5);   // Throws exception
        } catch (IllegalArgumentException e) {
            System.out.println("Validation error: " + e.getMessage());
        }
    }
}
```

throws Keyword

```
import java.io.*;

public class ThrowsExample {
    // Declare that this method may throw checked exceptions
     public void readFile(String filename) throws FileNotFoundException,
IOException {
                    BufferedReader reader = new BufferedReader(new
FileReader(filename));
        String line = reader.readLine();
        System.out.println(line);
        reader.close();
    }

    // Caller must handle or declare the exception
    public static void main(String[] args) {
        ThrowsExample example = new ThrowsExample();

        try {
            example.readFile("data.txt");
        } catch (FileNotFoundException e) {
            System.out.println("File not found!");
        } catch (IOException e) {
            System.out.println("IO Error: " + e.getMessage());
        }
    }
}
```

6.4 Custom Exceptions

Definition

A **custom exception** is a user-defined exception class that extends Exception (checked) or RuntimeException (unchecked). Custom exceptions make error handling more meaningful and domain-specific.

```
// Custom checked exception
public class InsufficientFundsException extends Exception {
    private double amount;
    private double balance;

    public InsufficientFundsException(double amount, double balance) {
        super("Insufficient funds: requested $" + amount
            + ", available $" + balance);
        this.amount = amount;
        this.balance = balance;
    }

    public double getAmount() { return amount; }
    public double getBalance() { return balance; }
    public double getShortfall() { return amount - balance; }
}

// Custom unchecked exception
public class InvalidEmailException extends RuntimeException {
    public InvalidEmailException(String email) {
        super("Invalid email format: " + email);
    }
}

// Bank account using custom exception
public class BankAccount {
    private double balance;

                public    void    withdraw(double    amount)    throws
InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException(amount, balance);
        }
        balance -= amount;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
                    throw  new  IllegalArgumentException("Amount  must  be
positive");
        }
        balance += amount;
    }

    public static void main(String[] args) {
        BankAccount account = new BankAccount();
```

Chapter 7

Generics

Generics enable types (classes and interfaces) to be parameters when defining classes, interfaces, and methods. They provide compile-time type safety and eliminate the need for explicit type casting.

7.1 Generic Classes

> **Definition**
>
> **Generics** allow you to create classes, interfaces, and methods that work with different data types while providing compile-time type checking. The type parameter is specified inside angle brackets `<T>`.

```
// Generic class with type parameter T
public class Box<T> {
    private T content;

    public void setContent(T content) {
        this.content = content;
    }

    public T getContent() {
        return content;
    }

    public boolean hasContent() {
        return content ≠ null;
    }

    public static void main(String[] args) {
        // Box for integers
        Box<Integer> intBox = new Box<>();
        intBox.setContent(100);
        int value = intBox.getContent();  // No casting needed
        System.out.println("Integer: " + value);

        // Box for strings
        Box<String> strBox = new Box<>();
        strBox.setContent("Hello Generics!");
        String text = strBox.getContent();
        System.out.println("String: " + text);

        // Box for custom objects
        Box<Person> personBox = new Box<>();
        personBox.setContent(new Person("Alice", 25));
                                    System.out.println("Person:     "     +
personBox.getContent().getName());
    }
}

class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
```

Multiple Type Parameters

```
// Generic class with multiple type parameters
public class Pair<K, V> {
    private K key;
    private V value;

    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() { return key; }
    public V getValue() { return value; }
    public void setValue(V value) { this.value = value; }

    @Override
    public String toString() {
        return "(" + key + "=" + value + ")";
    }

    public static void main(String[] args) {
        Pair<String, Integer> ageMap = new Pair<>("Alice", 25);
        System.out.println(ageMap);  // (Alice=25)

        Pair<Integer, String> idMap = new Pair<>(1, "First");
        System.out.println(idMap);   // (1=First)
    }
}
```

7.2 Generic Methods

```
public class GenericMethods {

    // Generic method with type parameter T
    public static <T> void printArray(T[] array) {
        for (T element : array) {
            System.out.print(element + " ");
        }
        System.out.println();
    }

    // Generic method that returns T
    public static <T> T getFirst(T[] array) {
        return array.length > 0 ? array[0] : null;
    }

    // Generic method with multiple type parameters
    public static <T, U> void printPair(T first, U second) {
                    System.out.println("First:  "  +  first  +  "  ("  +
first.getClass().getSimpleName() + ")");
                   System.out.println("Second:  "  +  second  +  "  ("  +
second.getClass().getSimpleName() + ")");
    }

    // Generic method in non-generic class
    public static void main(String[] args) {
        Integer[] intArray = {1, 2, 3, 4, 5};
        String[] strArray = {"Hello", "World"};

        printArray(intArray);  // 1 2 3 4 5
        printArray(strArray);  // Hello World

        Integer first = getFirst(intArray);
        System.out.println("First: " + first);

        printPair("Age", 25);
        printPair(100, 3.14);
    }
}
```

7.3 Bounded Type Parameters

> **Definition**
>
> A **bounded type parameter** restricts the types that can be used as type arguments. It ensures the type parameter extends a specific class or implements specific interfaces.

```
// T must be a subclass of Number
public class NumericBox<T extends Number> {
    private T value;

    public NumericBox(T value) {
        this.value = value;
    }

    public double getDoubleValue() {
            return value.doubleValue();   // Available because T extends
Number
    }

    public boolean isGreaterThan(T other) {
        return value.doubleValue() > other.doubleValue();
    }

    public static void main(String[] args) {
        NumericBox<Integer> intBox = new NumericBox<>(100);
        NumericBox<Double> doubleBox = new NumericBox<>(3.14);
        NumericBox<Float> floatBox = new NumericBox<>(2.5f);
          // NumericBox<String> strBox = new NumericBox<>("hello");   //
ERROR!

        System.out.println(intBox.getDoubleValue());
        System.out.println(doubleBox.isGreaterThan(floatBox));
    }
}
```

```
// Multiple bounds: T must extend a class AND implement interfaces
// Class must come first, then interfaces
public class SortableContainer<T extends Comparable<T> & Cloneable> {
    private T value;

    public SortableContainer(T value) {
        this.value = value;
    }

    public int compareTo(T other) {
        return value.compareTo(other);
    }
}
```

7.4 Wildcards

Definition

A **wildcard** (`?`) represents an unknown type. It provides flexibility when working with generic types, especially in method parameters. There are three types: unbounded, upper-bounded, and lower-bounded wildcards.

```
import java.util.*;

public class Wildcards {

    // Unbounded wildcard - accepts any type
    public static void printList(List<?> list) {
        for (Object obj : list) {
            System.out.print(obj + " ");
        }
        System.out.println();
    }

    // Upper bounded wildcard - accepts T or subclasses of T
    public static double sumNumbers(List<? extends Number> list) {
        double sum = 0;
        for (Number num : list) {
            sum += num.doubleValue();
        }
        return sum;
    }

    // Lower bounded wildcard - accepts T or superclasses of T
    public static void addIntegers(List<? super Integer> list) {
        list.add(10);
        list.add(20);
        list.add(30);
    }

    public static void main(String[] args) {
        // Unbounded wildcard
        List<String> strings = Arrays.asList("A", "B", "C");
        List<Integer> integers = Arrays.asList(1, 2, 3);
        printList(strings);
        printList(integers);

        // Upper bounded wildcard
        List<Integer> intList = Arrays.asList(10, 20, 30);
        List<Double> doubleList = Arrays.asList(1.5, 2.5, 3.5);
        System.out.println("Sum of ints: " + sumNumbers(intList));
        System.out.println("Sum of doubles: " + sumNumbers(doubleList));

        // Lower bounded wildcard
        List<Number> numbers = new ArrayList<>();
        addIntegers(numbers);
        System.out.println("Numbers: " + numbers);
```

Wildcard Type Summary

Table 7-1: Java Generic Wildcards

Wildcard	Syntax	Meaning	Use Case
Unbounded	`<?>`	Any type	Read-only operations
Upper bounded	`<? extends T>`	T or subclass of T	Reading values of type T
Lower bounded	`<? super T>`	T or superclass of T	Writing values of type T

PECS Principle

PECS: Producer Extends, Consumer Super

- Use `<? extends T>` when the generic produces T values (you read from it)
- Use `<? super T>` when the generic consumes T values (you write to it)

Chapter 8

Lambda Expressions and Functional Interfaces

Introduced in Java 8, lambda expressions and functional interfaces bring functional programming capabilities to Java, enabling more concise and readable code for operations on collections and event handling.

8.1 Functional Interfaces

> **Definition**
>
> A **functional interface** is an interface with exactly one abstract method (SAM — Single Abstract Method). It can have default and static methods. The `@FunctionalInterface` annotation ensures the interface meets this requirement.

```
@FunctionalInterface
public interface Calculator {
    int operate(int a, int b);  // Single abstract method

    // Default methods are allowed
    default void printResult(int result) {
        System.out.println("Result: " + result);
    }

    // Static methods are allowed
    static void describe() {
        System.out.println("Performs calculations on two integers");
    }
}
```

Built-in Functional Interfaces

Table 8-1: Java Built-in Functional Interfaces (java.util.function)

Interface	Method	Description	Example
`Predicate<T>`	`test(T) → boolean`	Tests a condition	`s -> s.length() > 5`
`Function<T,R>`	`apply(T) → R`	Transforms T to R	`s -> s.length()`
`Consumer<T>`	`accept(T) → void`	Consumes a value	`s -> System.out.println(s)`
`Supplier<T>`	`get() → T`	Supplies a value	`() -> new Random().nextInt()`
`UnaryOperator<T>`	`apply(T) → T`	Unary operation on T	`x -> x * x`
`BinaryOperator<T>`	`apply(T,T) → T`	Binary operation on T	`(a, b) -> a + b`
`BiPredicate<T,U>`	`test(T,U) → boolean`	Tests with two inputs	`(s, n) -> s.length() > n`
`BiFunction<T,U,R>`	`apply(T,U) → R`	Transforms two inputs	`(s, n) -> s.repeat(n)`
`BiConsumer<T,U>`	`accept(T,U) → void`	Consumes two values	`(k, v) -> map.put(k, v)`

```
import java.util.function.*;

public class BuiltInFunctionalInterfaces {
    public static void main(String[] args) {
        // Predicate - tests a condition
        Predicate<Integer> isEven = n -> n % 2 == 0;
        System.out.println("Is 4 even? " + isEven.test(4));  // true

        // Function - transforms input to output
        Function<String, Integer> length = s -> s.length();
        System.out.println("Length: " + length.apply("Hello"));  // 5

        // Consumer - accepts a value, returns nothing
        Consumer<String> printer = s -> System.out.println(s);
        printer.accept("Hello Consumer!");

        // Supplier - provides a value
        Supplier<Double> random = () -> Math.random();
        System.out.println("Random: " + random.get());

        // Combining predicates
        Predicate<Integer> isPositive = n -> n > 0;
        Predicate<Integer> isLessThan100 = n -> n < 100;
        Predicate<Integer> isValid = isPositive.and(isLessThan100);
        System.out.println("Is 50 valid? " + isValid.test(50));  // true
    }
}
```

8.2 Lambda Syntax

> **Definition**
>
> A **lambda expression** is an anonymous function that provides a concise way to implement a functional interface. It consists of parameters, an arrow token `->`, and a body.

```
// Syntax variations of lambda expressions

// Full syntax with types
Calculator add = (int a, int b) -> { return a + b; };

// Type inference (compiler deduces types)
Calculator subtract = (a, b) -> { return a - b; };

// Single expression (return and braces optional)
Calculator multiply = (a, b) -> a * b;

// Single parameter (parentheses optional)
Function<String, Integer> length = s -> s.length();

// No parameters
Runnable task = () -> System.out.println("Running!");

// Multi-line body
Calculator divide = (a, b) -> {
    if (b == 0) {
        throw new ArithmeticException("Cannot divide by zero");
    }
    return a / b;
};
```

```
import java.util.*;

public class LambdaExamples {
    public static void main(String[] args) {
          List<String> names = Arrays.asList("Alice", "Bob", "Charlie",
"David");

        // Sorting with lambda
        names.sort((a, b) -> a.length() - b.length());
        System.out.println("Sorted by length: " + names);

        // forEach with lambda
        names.forEach(name -> System.out.println(name));

        // Filtering with Predicate
        names.removeIf(name -> name.length() < 4);
        System.out.println("After filter: " + names);

        // Using Comparator.comparing
        List<Person> people = Arrays.asList(
            new Person("Alice", 25),
            new Person("Bob", 30),
            new Person("Charlie", 20)
        );

        people.sort(Comparator.comparing(p -> p.getName()));
        people.sort(Comparator.comparing(Person::getAge).reversed());
    }
}
```

8.3 Method References

> **Definition**
>
> A **method reference** is a shorthand notation for a lambda expression that calls an existing method. It uses the `::` operator and is more readable when the lambda simply calls a method.

Types of Method References

Table 8-2: Method Reference Types

Type	Syntax	Equivalent Lambda	Example
Static method	`ClassName::staticMethod`	`(args) -> ClassName.staticMethod(args)`	`Integer::parseInt`
Instance method (object)	`object::method`	`(args) -> object.method(args)`	`scanner::nextLine`
Instance method (class)	`ClassName::method`	`(obj, args) -> obj.method(args)`	`String::length`
Constructor	`ClassName::new`	`(args) -> new ClassName(args)`	`ArrayList::new`

```
import java.util.*;
import java.util.function.*;
import java.util.stream.*;

public class MethodReferenceDemo {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("alice", "bob", "charlie");

        // Static method reference
        names.stream()
             .map(String::toUpperCase)           // s -> s.toUpperCase()
                      .forEach(System.out::println);            //  s  ->
System.out.println(s)

        // Constructor reference
        List<String> copied = names.stream()
            .collect(Collectors.toCollection(ArrayList::new));

        // Using constructor reference for Supplier
        Supplier<List<String>> listFactory = ArrayList::new;
        List<String> newList = listFactory.get();

        // Instance method on a specific object
        String prefix = "Mr. ";
        names.forEach(name -> System.out.println(prefix + name));

        // Comparing with method reference
        List<Person> people = Arrays.asList(
            new Person("Alice", 25),
            new Person("Bob", 30)
        );

        // Method reference to instance method of arbitrary object
        people.sort(Comparator.comparing(Person::getName));
        people.sort(Comparator.comparingInt(Person::getAge));

        // Function composition
        Function<String, String> trim = String::trim;
        Function<String, String> toUpper = String::toUpperCase;
        Function<String, String> trimAndUpper = trim.andThen(toUpper);

        System.out.println(trimAndUpper.apply("  hello  "));  // HELLO
    }
}
```

Chapter 9

Streams API

The Streams API, introduced in Java 8, provides a functional approach to processing collections of objects. It allows you to perform complex data manipulations using a declarative, pipeline-style syntax.

9.1 Creating Streams

> **Definition**
>
> A **Stream** is a sequence of elements that supports various operations for computing results. Streams do not store data; they process data from a source (like a collection, array, or I/O channel) on-demand.

```
import java.util.*;
import java.util.stream.*;

public class CreatingStreams {
    public static void main(String[] args) {
        // From a Collection
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
        Stream<Integer> stream1 = numbers.stream();

        // From an Array
        String[] fruits = {"Apple", "Banana", "Cherry"};
        Stream<String> stream2 = Arrays.stream(fruits);

        // Using Stream.of()
        Stream<Integer> stream3 = Stream.of(1, 2, 3, 4, 5);

        // Empty stream
        Stream<String> emptyStream = Stream.empty();

        // Stream builder
        Stream<String> builtStream = Stream.<String>builder()
            .add("Java")
            .add("Python")
            .add("C++")
            .build();

        // Generate infinite stream (use limit!)
                                    Stream<Double>     randomStream     =
Stream.generate(Math::random).limit(5);

        // Iterate
             Stream<Integer>  evenNumbers  =  Stream.iterate(0,  n  ->  n  +
2).limit(10);
        // Java 9+: iterate with predicate
        Stream<Integer> finite = Stream.iterate(1, n -> n <= 100, n -> n
* 2);

        // Stream of primitives (more efficient)
        IntStream intStream = IntStream.range(1, 10);      // 1 to 9
         IntStream rangeClosed = IntStream.rangeClosed(1, 10);   // 1 to
10
        LongStream longStream = LongStream.range(1, 100);
        DoubleStream doubleStream = DoubleStream.of(1.5, 2.5, 3.5);

        // From a file (throws IOException)
        // Stream<String> lines = Files.lines(Path.of("file.txt"));
```

9.2 Intermediate Operations

> **Definition**
>
> **Intermediate operations** transform a stream into another stream. They are lazy — they do not execute until a terminal operation is called. Intermediate operations can be chained together.

```
import java.util.*;
import java.util.stream.*;

public class IntermediateOperations {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9,
10);
          List<String> names = Arrays.asList("Alice", "Bob", "Charlie",
"David", "Eve");

        // filter() - keeps elements matching predicate
        List<Integer> evens = numbers.stream()
            .filter(n -> n % 2 == 0)
            .collect(Collectors.toList());
         System.out.println("Even numbers: " + evens);  // [2, 4, 6, 8,
10]

        // map() - transforms each element
        List<Integer> squared = numbers.stream()
            .map(n -> n * n)
            .collect(Collectors.toList());
        System.out.println("Squared: " + squared);  // [1, 4, 9, 16, 25,
 ... ]

        // flatMap() - flattens nested structures
        List<List<Integer>> nested = Arrays.asList(
            Arrays.asList(1, 2),
            Arrays.asList(3, 4),
            Arrays.asList(5, 6)
        );
        List<Integer> flat = nested.stream()
            .flatMap(List::stream)
            .collect(Collectors.toList());
        System.out.println("Flattened: " + flat);  // [1, 2, 3, 4, 5, 6]

        // distinct() - removes duplicates
        List<Integer> unique = Stream.of(1, 2, 2, 3, 3, 3)
            .distinct()
            .collect(Collectors.toList());
        System.out.println("Unique: " + unique);  // [1, 2, 3]

        // sorted() - sorts elements
        List<Integer> sortedDesc = numbers.stream()
            .sorted(Comparator.reverseOrder())
            .collect(Collectors.toList());
        System.out.println("Descending: " + sortedDesc);
```

9.3 Terminal Operations

> **Definition**
>
> **Terminal operations** produce a result or side-effect and close the stream. After a terminal operation, the stream cannot be reused. They trigger the execution of all lazy intermediate operations.

```
import java.util.*;
import java.util.stream.*;

public class TerminalOperations {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9,
10);

        // forEach() - performs action on each element
        numbers.stream()
            .filter(n -> n % 2 == 0)
            .forEach(n -> System.out.print(n + " "));
        System.out.println();

        // collect() - gathers elements into a collection
        List<Integer> doubled = numbers.stream()
            .map(n -> n * 2)
            .collect(Collectors.toList());

        Set<Integer> set = numbers.stream()
            .collect(Collectors.toSet());

        // reduce() - combines elements into a single value
        int sum = numbers.stream()
            .reduce(0, (a, b) -> a + b);
        System.out.println("Sum: " + sum);

        int product = numbers.stream()
            .reduce(1, (a, b) -> a * b);
        System.out.println("Product: " + product);

        // Without identity (returns Optional)
        Optional<Integer> max = numbers.stream()
            .reduce((a, b) -> a > b ? a : b);
        max.ifPresent(m -> System.out.println("Max: " + m));

        // count()
        long count = numbers.stream().count();
        System.out.println("Count: " + count);

        // anyMatch(), allMatch(), noneMatch()
        boolean hasEven = numbers.stream().anyMatch(n -> n % 2 == 0);
        boolean allPositive = numbers.stream().allMatch(n -> n > 0);
        boolean noNegative = numbers.stream().noneMatch(n -> n < 0);

        // findFirst() and findAny()
```

9.4 Collectors

```
import java.util.*;
import java.util.stream.*;

public class CollectorsDemo {
    public static void main(String[] args) {
        List<Person> people = Arrays.asList(
            new Person("Alice", 25, "Engineering"),
            new Person("Bob", 30, "Sales"),
            new Person("Charlie", 25, "Engineering"),
            new Person("David", 35, "Sales"),
            new Person("Eve", 28, "HR")
        );

        // toList() and toSet()
        List<String> names = people.stream()
            .map(Person::getName)
            .collect(Collectors.toList());

        // joining() - concatenate strings
        String joined = people.stream()
            .map(Person::getName)
            .collect(Collectors.joining(", "));
         System.out.println("Names: " + joined);  // Alice, Bob, Charlie,
David, Eve

        // joining() with prefix and suffix
        String formatted = people.stream()
            .map(Person::getName)
            .collect(Collectors.joining(", ", "[", "]"));
         System.out.println(formatted);  // [Alice, Bob, Charlie, David,
Eve]

        // groupingBy()
        Map<String, List<Person>> byDept = people.stream()
            .collect(Collectors.groupingBy(Person::getDepartment));
        System.out.println("By department: " + byDept.keySet());

        // partitioningBy()
        Map<Boolean, List<Person>> partitioned = people.stream()
            .collect(Collectors.partitioningBy(p -> p.getAge() >= 30));
        System.out.println("Seniors: " + partitioned.get(true).size());

        // counting()
        Map<String, Long> deptCount = people.stream()
            .collect(Collectors.groupingBy(
                Person::getDepartment,
```

Stream Best Practices

- Streams are one-time use — once consumed, they cannot be reused
- Use primitive streams (IntStream, LongStream) for better performance with numbers
- Prefer method references over lambdas for readability when possible
- Use parallelStream() cautiously — not always faster due to overhead
- Avoid modifying the source collection while streaming

Chapter 10

Java Collections Framework

The Java Collections Framework (JCF) is a unified architecture for representing and manipulating collections. It reduces programming effort, increases performance, and enables interoperability between unrelated APIs.

10.1 Collection Hierarchy

> **Definition**
>
> The **Collections Framework** is a set of classes and interfaces that implement commonly reusable collection data structures. It is located in the `java.util` package.

Core Interfaces

Table 10-1: Java Collections Framework Core Interfaces

Interface	Description	Key Implementations
`Collection`	Root interface for all collections	--
`List`	Ordered, allows duplicates, indexed	ArrayList, LinkedList, Vector
`Set`	Unordered, no duplicates	HashSet, LinkedHashSet, TreeSet
`Queue`	FIFO (First-In-First-Out)	LinkedList, PriorityQueue, ArrayDeque
`Deque`	Double-ended queue	ArrayDeque, LinkedList
`Map`	Key-value pairs (not a Collection)	HashMap, LinkedHashMap, TreeMap

```
import java.util.*;

public class CollectionHierarchy {
    public static void main(String[] args) {
        // All collections implement the Collection interface
        Collection<String> collection = new ArrayList<>();
        collection.add("A");
        collection.add("B");

        // Common Collection methods
        System.out.println("Size: " + collection.size());
        System.out.println("Contains A? " + collection.contains("A"));
        System.out.println("Is empty? " + collection.isEmpty());

        collection.remove("A");
        collection.clear();

        // Add all elements from another collection
        List<String> list = Arrays.asList("X", "Y", "Z");
        collection.addAll(list);

        // Convert to array
        String[] array = collection.toArray(new String[0]);

        // Iterator
        Iterator<String> iterator = collection.iterator();
        while (iterator.hasNext()) {
            System.out.println(iterator.next());
        }
    }
}
```

10.2 List Interface

> **Definition**
>
> A **List** is an ordered collection that allows duplicate elements. Elements can be accessed by their integer index (position). Lists maintain insertion order.

```
import java.util.*;

public class ListInterface {
    public static void main(String[] args) {
        // ===== ArrayList =====
              // Backed by dynamic array, fast random access, slow
insertion/deletion in middle
        List<String> arrayList = new ArrayList<>();
        arrayList.add("First");
        arrayList.add("Second");
        arrayList.add(1, "Inserted");  // Insert at index
        arrayList.add("Duplicate");
        arrayList.add("Duplicate");    // Duplicates allowed

        System.out.println("ArrayList: " + arrayList);
        System.out.println("Element at 0: " + arrayList.get(0));
                      System.out.println("Index   of   'Second':   "   +
arrayList.indexOf("Second"));

        // ===== LinkedList =====
           // Doubly-linked list, fast insertion/deletion, slow random
access
        List<String> linkedList = new LinkedList<>();
        linkedList.add("Alpha");
        linkedList.add("Beta");
        linkedList.add("Gamma");

        // LinkedList as Deque (double-ended queue)
        LinkedList<String> deque = new LinkedList<>();
        deque.addFirst("First");
        deque.addLast("Last");
        System.out.println("First: " + deque.getFirst());
        System.out.println("Last: " + deque.getLast());

        // ===== Vector (legacy, thread-safe) =====
        List<String> vector = new Vector<>();
        vector.add("Item1");
        vector.add("Item2");

        // ===== List Methods =====
          List<Integer> numbers = new ArrayList<>(Arrays.asList(10, 20,
30, 40, 50));

        // Sublist (view of original list)
        List<Integer> sub = numbers.subList(1, 4);
        System.out.println("Sublist: " + sub);  // [20, 30, 40]
```

ArrayList vs LinkedList

Table 10-2: ArrayList vs LinkedList Comparison

Operation	ArrayList	LinkedList
get(index)	O(1) — constant	O(n) — linear
add(end)	O(1) amortized	O(1)
add(middle)	O(n) — shifts elements	O(1) — with iterator
remove(middle)	O(n) — shifts elements	O(1) — with iterator
Memory	Less overhead	More overhead (node objects)
Use when	More reads, fewer modifications	Frequent insertions/deletions

10.3 Set Interface

> **Definition**
>
> A **Set** is a collection that does not allow duplicate elements. It models the mathematical set abstraction. Sets do not guarantee any specific order (except sorted variants).

```
import java.util.*;

public class SetInterface {
    public static void main(String[] args) {
        // ===== HashSet =====
        // Unordered, uses hash table, O(1) operations
        Set<String> hashSet = new HashSet<>();
        hashSet.add("Banana");
        hashSet.add("Apple");
        hashSet.add("Cherry");
        hashSet.add("Apple");  // Duplicate, ignored
          System.out.println("HashSet: " + hashSet);   // No guaranteed
order

        // ===== LinkedHashSet =====
        // Maintains insertion order
        Set<String> linkedHashSet = new LinkedHashSet<>();
        linkedHashSet.add("Banana");
        linkedHashSet.add("Apple");
        linkedHashSet.add("Cherry");
            System.out.println("LinkedHashSet: " + linkedHashSet);   //
Insertion order

        // ===== TreeSet =====
        // Sorted (natural order or custom comparator)
        Set<String> treeSet = new TreeSet<>();
        treeSet.add("Banana");
        treeSet.add("Apple");
        treeSet.add("Cherry");
         System.out.println("TreeSet: " + treeSet);   // [Apple, Banana,
Cherry]

        // TreeSet with custom comparator
                        Set<String>   descending   =   new   TreeSet<>
(Collections.reverseOrder());
        descending.addAll(Arrays.asList("A", "B", "C"));
        System.out.println("Descending: " + descending);  // [C, B, A]

        // Set operations
        Set<Integer> set1 = new HashSet<>(Arrays.asList(1, 2, 3, 4));
        Set<Integer> set2 = new HashSet<>(Arrays.asList(3, 4, 5, 6));

        // Union
        Set<Integer> union = new HashSet<>(set1);
        union.addAll(set2);
        System.out.println("Union: " + union);  // [1, 2, 3, 4, 5, 6]
```

10.4 Queue Interface

> **Definition**
>
> A **Queue** is a collection designed for holding elements before processing. It typically orders elements in FIFO (First-In-First-Out) manner. The head of the queue is the element that has been on the queue the longest.

```
import java.util.*;

public class QueueInterface {
    public static void main(String[] args) {
        // ===== LinkedList as Queue =====
        Queue<String> queue = new LinkedList<>();

        // Enqueue elements
           queue.offer("Alice");    // Add (preferred, returns false if
full)
        queue.add("Bob");       // Add (throws exception if full)
        queue.offer("Charlie");

        System.out.println("Queue: " + queue);

        // Peek at head (does not remove)
        System.out.println("Head: " + queue.peek());    // Alice
        System.out.println("Head (element): " + queue.element());

        // Dequeue (remove and return head)
        System.out.println("Removed: " + queue.poll());  // Alice
        System.out.println("Removed: " + queue.remove()); // Bob

        // ===== PriorityQueue =====
        // Orders elements by natural ordering or custom comparator
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        pq.offer(30);
        pq.offer(10);
        pq.offer(20);
        pq.offer(5);

        System.out.println("\nPriorityQueue (min-heap):");
        while (!pq.isEmpty()) {
            System.out.print(pq.poll() + " ");  // 5 10 20 30
        }
        System.out.println();

        // PriorityQueue with custom comparator (max-heap)
                PriorityQueue<Integer>  maxHeap  =  new  PriorityQueue<>
(Collections.reverseOrder());
        maxHeap.addAll(Arrays.asList(30, 10, 20, 5));

        System.out.println("Max-heap:");
        while (!maxHeap.isEmpty()) {
            System.out.print(maxHeap.poll() + " ");  // 30 20 10 5
        }
```

10.5 Map Interface

> **Definition**
>
> A **Map** is an object that maps keys to values. Each key maps to exactly one value. Keys must be unique, but values can be duplicated. Map does not extend Collection.

```
import java.util.*;

public class MapInterface {
    public static void main(String[] args) {
        // ===== HashMap =====
        // Unordered, uses hash table, O(1) operations
        Map<String, Integer> hashMap = new HashMap<>();
        hashMap.put("Alice", 25);
        hashMap.put("Bob", 30);
        hashMap.put("Charlie", 25);
        hashMap.put("Alice", 26);  // Overwrites existing key

        System.out.println("HashMap: " + hashMap);
        System.out.println("Alice's age: " + hashMap.get("Alice"));
                        System.out.println("Contains   'Bob'?   "   +
hashMap.containsKey("Bob"));
                       System.out.println("Contains   age   30?   "   +
hashMap.containsValue(30));

        // ===== LinkedHashMap =====
        // Maintains insertion order
        Map<String, Integer> linkedHashMap = new LinkedHashMap<>();
        linkedHashMap.put("Third", 3);
        linkedHashMap.put("First", 1);
        linkedHashMap.put("Second", 2);
        System.out.println("\nLinkedHashMap: " + linkedHashMap);

        // ===== TreeMap =====
        // Sorted by key (natural order or comparator)
        Map<String, Integer> treeMap = new TreeMap<>();
        treeMap.put("Charlie", 3);
        treeMap.put("Alice", 1);
        treeMap.put("Bob", 2);
        System.out.println("TreeMap: " + treeMap);  // Sorted by key

        // ===== Iterating over Map =====
        // Method 1: entrySet()
        System.out.println("\nUsing entrySet():");
        for (Map.Entry<String, Integer> entry : hashMap.entrySet()) {
                        System.out.println(entry.getKey()  +  "  →  "  +
entry.getValue());
        }

        // Method 2: forEach (Java 8)
        System.out.println("\nUsing forEach():");
        hashMap.forEach((key, value) → System.out.println(key + " = " +
```

10.6 Choosing the Right Collection

Table 10-3: Collection Selection Guide

Need	Use	Implementation
Fast access by index	`List`	`ArrayList`
Frequent insert/delete at ends	`Deque`	`ArrayDeque`
Frequent insert/delete in middle	`List`	`LinkedList`
No duplicates, fast lookup	`Set`	`HashSet`
No duplicates, sorted	`SortedSet`	`TreeSet`
No duplicates, insertion order	`Set`	`LinkedHashSet`
Key-value lookup	`Map`	`HashMap`
Sorted key-value pairs	`SortedMap`	`TreeMap`
FIFO queue	`Queue`	`ArrayDeque`
Priority processing	`Queue`	`PriorityQueue`
Thread-safe access	Concurrent collections	`ConcurrentHashMap`, `CopyOnWriteArrayList`

Best Practices

- Program to interfaces: declare variables as `List<T>`, not `ArrayList<T>`
- Use diamond operator `<>` for type inference (Java 7+)
- Consider `Collections.unmodifiableXXX()` for immutable collections
- Use `Arrays.asList()` for fixed-size lists, `List.of()` for immutable lists (Java 9+)
- Prefer `Map.of()`, `Set.of()`, `List.of()` for small, immutable collections (Java 9+)
- Specify initial capacity for ArrayList/HashMap when size is known

Chapter 11

Data Structures

Data structures are fundamental concepts in computer science that define how data is organized, stored, and accessed. Understanding data structures is essential for writing efficient algorithms.

11.1 Linked List

> **Definition: Linked List**
>
> A **Linked List** is a linear data structure where elements (nodes) are connected via references. Each node contains data and a reference to the next node. Unlike arrays, linked lists do not require contiguous memory allocation.

Singly Linked List

```
// Node class
class ListNode {
    int data;
    ListNode next;

    public ListNode(int data) {
        this.data = data;
        this.next = null;
    }
}

// Singly Linked List implementation
public class SinglyLinkedList {
    private ListNode head;
    private int size;

    public SinglyLinkedList() {
        head = null;
        size = 0;
    }

    // Add at the beginning: O(1)
    public void addFirst(int data) {
        ListNode newNode = new ListNode(data);
        newNode.next = head;
        head = newNode;
        size++;
    }

    // Add at the end: O(n)
    public void addLast(int data) {
        ListNode newNode = new ListNode(data);
        if (head == null) {
            head = newNode;
        } else {
            ListNode current = head;
            while (current.next != null) {
                current = current.next;
            }
            current.next = newNode;
        }
        size++;
    }

    // Add at index: O(n)
    public void add(int index, int data) {
```

Doubly Linked List

```
class DoublyNode {
    int data;
    DoublyNode prev;
    DoublyNode next;

    public DoublyNode(int data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}

public class DoublyLinkedList {
    private DoublyNode head;
    private DoublyNode tail;

    // Add at end: O(1)
    public void addLast(int data) {
        DoublyNode newNode = new DoublyNode(data);
        if (head == null) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }

    // Add at beginning: O(1)
    public void addFirst(int data) {
        DoublyNode newNode = new DoublyNode(data);
        if (head == null) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }

    // Remove from end: O(1)
    public void removeLast() {
        if (tail == null) return;
        if (head == tail) {
            head = tail = null;
        } else {
```

11.2 Stack

> **Definition: Stack**
>
> A **Stack** is a linear data structure that follows the **LIFO** (Last-In-First-Out) principle. The last element added is the first one to be removed. Operations are performed at one end called the **top**.

```
// Stack implementation using array
public class ArrayStack<T> {
    private Object[] elements;
    private int size;
    private static final int DEFAULT_CAPACITY = 10;

    public ArrayStack() {
        elements = new Object[DEFAULT_CAPACITY];
        size = 0;
    }

    // Push: O(1) amortized
    public void push(T item) {
        ensureCapacity();
        elements[size++] = item;
    }

    // Pop: O(1)
    @SuppressWarnings("unchecked")
    public T pop() {
        if (isEmpty()) {
            throw new EmptyStackException();
        }
        T item = (T) elements[--size];
        elements[size] = null;  // Prevent memory leak
        return item;
    }

    // Peek: O(1)
    @SuppressWarnings("unchecked")
    public T peek() {
        if (isEmpty()) {
            throw new EmptyStackException();
        }
        return (T) elements[size - 1];
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public int size() {
        return size;
    }

    private void ensureCapacity() {
```

```
// Check balanced parentheses using Stack
public class BalancedParentheses {
    public static boolean isBalanced(String expression) {
        Stack<Character> stack = new Stack<>();

        for (char ch : expression.toCharArray()) {
            if (ch == '(' || ch == '{' || ch == '[') {
                stack.push(ch);
            } else if (ch == ')' || ch == '}' || ch == ']') {
                if (stack.isEmpty()) return false;

                char open = stack.pop();
                if ((ch == ')' && open != '(') ||
                    (ch == '}' && open != '{') ||
                    (ch == ']' && open != '[')) {
                    return false;
                }
            }
        }
        return stack.isEmpty();
    }

    public static void main(String[] args) {
        System.out.println(isBalanced("{[()]}"));      // true
        System.out.println(isBalanced("{[(])}"));      // false
        System.out.println(isBalanced("{{[[(())]]}}")); // true
    }
}
```

11.3 Queue

> **Definition: Queue**
>
> A **Queue** is a linear data structure that follows the **FIFO** (First-In-First-Out) principle. Elements are added at the **rear** (enqueue) and removed from the **front** (dequeue).

```
// Circular Queue implementation
public class CircularQueue<T> {
    private Object[] elements;
    private int front;
    private int rear;
    private int size;
    private int capacity;

    public CircularQueue(int capacity) {
        this.capacity = capacity;
        this.elements = new Object[capacity];
        this.front = 0;
        this.rear = -1;
        this.size = 0;
    }

    // Enqueue: O(1)
    public void enqueue(T item) {
        if (isFull()) {
            throw new IllegalStateException("Queue is full");
        }
        rear = (rear + 1) % capacity;
        elements[rear] = item;
        size++;
    }

    // Dequeue: O(1)
    @SuppressWarnings("unchecked")
    public T dequeue() {
        if (isEmpty()) {
            throw new IllegalStateException("Queue is empty");
        }
        T item = (T) elements[front];
        elements[front] = null;
        front = (front + 1) % capacity;
        size--;
        return item;
    }

    @SuppressWarnings("unchecked")
    public T peek() {
        if (isEmpty()) {
            throw new IllegalStateException("Queue is empty");
        }
        return (T) elements[front];
    }
```

11.4 Binary Trees

> **Definition: Binary Tree**
>
> A **Binary Tree** is a hierarchical data structure where each node has at most two children, referred to as the **left child** and **right child**. The topmost node is called the **root**.

```
class TreeNode {
    int data;
    TreeNode left;
    TreeNode right;

    public TreeNode(int data) {
        this.data = data;
        this.left = null;
        this.right = null;
    }
}

public class BinaryTree {
    private TreeNode root;

    // Insert level-order (BFS)
    public void insert(int data) {
        if (root == null) {
            root = new TreeNode(data);
            return;
        }

        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);

        while (!queue.isEmpty()) {
            TreeNode current = queue.poll();

            if (current.left == null) {
                current.left = new TreeNode(data);
                return;
            } else if (current.right == null) {
                current.right = new TreeNode(data);
                return;
            }

            queue.offer(current.left);
            queue.offer(current.right);
        }
    }

    // Inorder Traversal (Left → Root → Right): O(n)
    public void inorder(TreeNode node) {
        if (node ≠ null) {
            inorder(node.left);
            System.out.print(node.data + " ");
```

11.5 Binary Search Tree

> **Definition: Binary Search Tree**
>
> A **Binary Search Tree (BST)** is a binary tree where for each node, all values in the left subtree are less than the node's value, and all values in the right subtree are greater. This property enables efficient search, insertion, and deletion operations.

```
public class BinarySearchTree {
    private TreeNode root;

    // Insert: O(h) where h is height, O(log n) for balanced tree
    public void insert(int data) {
        root = insertRecursive(root, data);
    }

    private TreeNode insertRecursive(TreeNode node, int data) {
        if (node == null) return new TreeNode(data);

        if (data < node.data) {
            node.left = insertRecursive(node.left, data);
        } else if (data > node.data) {
            node.right = insertRecursive(node.right, data);
        }
        // Duplicate values are ignored
        return node;
    }

    // Search: O(h)
    public boolean search(int data) {
        return searchRecursive(root, data);
    }

    private boolean searchRecursive(TreeNode node, int data) {
        if (node == null) return false;
        if (data == node.data) return true;

        return data < node.data
            ? searchRecursive(node.left, data)
            : searchRecursive(node.right, data);
    }

    // Find minimum
    public int findMin() {
        if (root == null) throw new IllegalStateException("Tree is
empty");
        TreeNode current = root;
        while (current.left ≠ null) {
            current = current.left;
        }
        return current.data;
    }

    // Find maximum
```

11.6 Heaps and Priority Queues

> **Definition: Heap**
>
> A **Heap** is a complete binary tree that satisfies the heap property. In a **min-heap**, every parent node is smaller than or equal to its children. In a **max-heap**, every parent is greater than or equal to its children.

```
// Min Heap implementation
public class MinHeap {
    private int[] heap;
    private int size;
    private int capacity;

    public MinHeap(int capacity) {
        this.capacity = capacity;
        this.heap = new int[capacity];
        this.size = 0;
    }

    // Get parent/child indices
    private int parent(int i) { return (i - 1) / 2; }
    private int leftChild(int i) { return 2 * i + 1; }
    private int rightChild(int i) { return 2 * i + 2; }

    // Insert: O(log n)
    public void insert(int value) {
         if (size ≥ capacity) throw new IllegalStateException("Heap is
full");

        heap[size] = value;
        int current = size;
        size++;

        // Heapify up
        while (current > 0 && heap[current] < heap[parent(current)]) {
            swap(current, parent(current));
            current = parent(current);
        }
    }

    // Extract min (remove and return root): O(log n)
    public int extractMin() {
        if (size ≤ 0) throw new IllegalStateException("Heap is empty");

        int min = heap[0];
        heap[0] = heap[size - 1];
        size--;
        heapify(0);
        return min;
    }

    // Heapify down
    private void heapify(int i) {
```

11.7 Hash Tables

> **Definition: Hash Table**
>
> A **Hash Table** is a data structure that implements an associative array using a hash function to map keys to indices in an array. It provides O(1) average-case time complexity for insert, delete, and search operations.

```
// Simple HashMap implementation with chaining
public class SimpleHashMap<K, V> {
    private static class Entry<K, V> {
        K key;
        V value;
        Entry<K, V> next;

        public Entry(K key, V value) {
            this.key = key;
            this.value = value;
        }
    }

    private Entry<K, V>[] buckets;
    private int capacity = 16;
    private int size;

    @SuppressWarnings("unchecked")
    public SimpleHashMap() {
        buckets = new Entry[capacity];
    }

    private int getBucketIndex(K key) {
        return (key.hashCode() & 0x7fffffff) % capacity;
    }

    // Put: O(1) average
    public void put(K key, V value) {
        int index = getBucketIndex(key);
        Entry<K, V> head = buckets[index];

        // Check if key exists
        while (head != null) {
            if (head.key.equals(key)) {
                head.value = value;  // Update existing
                return;
            }
            head = head.next;
        }

        // Insert new entry at head
        size++;
        head = buckets[index];
        Entry<K, V> newEntry = new Entry<>(key, value);
        newEntry.next = head;
        buckets[index] = newEntry;
```

11.8 Graphs

> **Definition: Graph**
>
> A **Graph** is a non-linear data structure consisting of **vertices** (nodes) connected by **edges**. Graphs can be **directed** (edges have direction) or **undirected**, and **weighted** (edges have values) or **unweighted**.

```
import java.util.*;

// Adjacency List representation
public class Graph {
    private int vertices;
    private List<List<Integer>> adjList;

    public Graph(int vertices) {
        this.vertices = vertices;
        this.adjList = new ArrayList<>();
        for (int i = 0; i < vertices; i++) {
            adjList.add(new ArrayList<>());
        }
    }

    public void addEdge(int source, int destination) {
        adjList.get(source).add(destination);
        adjList.get(destination).add(source);  // Undirected
    }

    // BFS Traversal: O(V + E)
    public void bfs(int startVertex) {
        boolean[] visited = new boolean[vertices];
        Queue<Integer> queue = new LinkedList<>();

        visited[startVertex] = true;
        queue.offer(startVertex);

        System.out.print("BFS: ");
        while (!queue.isEmpty()) {
            int vertex = queue.poll();
            System.out.print(vertex + " ");

            for (int neighbor : adjList.get(vertex)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    queue.offer(neighbor);
                }
            }
        }
        System.out.println();
    }

    // DFS Traversal: O(V + E)
    public void dfs(int startVertex) {
        boolean[] visited = new boolean[vertices];
```

Chapter 12

Algorithms

Algorithms are step-by-step procedures for solving problems. Understanding common algorithms and their complexity is essential for writing efficient code.

12.1 Sorting Algorithms

> **Definition: Sorting Algorithm**
>
> A **sorting algorithm** arranges elements of a list in a specific order (typically ascending or descending). Sorting is fundamental to many computing tasks and enables efficient searching.

Bubble Sort

Bubble Sort — $O(n^2)$

Repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The largest unsorted element "bubbles up" to its correct position in each pass.

```java
public class BubbleSort {
    public static void bubbleSort(int[] arr) {
        int n = arr.length;

        for (int i = 0; i < n - 1; i++) {
            boolean swapped = false;

            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    // Swap arr[j] and arr[j+1]
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    swapped = true;
                }
            }

            // Optimization: if no swaps occurred, array is sorted
            if (!swapped) break;
        }
    }

    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};
        bubbleSort(arr);
        System.out.println("Sorted: " + Arrays.toString(arr));
        // Sorted: [11, 12, 22, 25, 34, 64, 90]
    }
}
```

Selection Sort

Selection Sort — $O(n^2)$

Finds the minimum element from the unsorted portion and places it at the beginning. Repeats this process for the remaining unsorted portion.

```java
public class SelectionSort {
    public static void selectionSort(int[] arr) {
        int n = arr.length;

        for (int i = 0; i < n - 1; i++) {
            int minIndex = i;

            // Find the minimum element in unsorted portion
            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[minIndex]) {
                    minIndex = j;
                }
            }

            // Swap the found minimum with the first element
            int temp = arr[minIndex];
            arr[minIndex] = arr[i];
            arr[i] = temp;
        }
    }
}
```

Insertion Sort

Insertion Sort — $O(n^2)$, $O(n)$ best case

Builds the final sorted array one item at a time. Takes each element and inserts it into its correct position in the already-sorted portion.

```java
public class InsertionSort {
    public static void insertionSort(int[] arr) {
        for (int i = 1; i < arr.length; i++) {
            int key = arr[i];  // Current element to insert
            int j = i - 1;

            // Shift elements greater than key to the right
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;  // Insert key at correct position
        }
    }
}
```

Merge Sort

Merge Sort — O(n log n)

A divide-and-conquer algorithm that divides the array into halves, recursively sorts each half, and then merges the sorted halves back together.

```
public class MergeSort {
    public static void mergeSort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;

            // Sort left and right halves
            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);

            // Merge the sorted halves
            merge(arr, left, mid, right);
        }
    }

    private static void merge(int[] arr, int left, int mid, int right) {
        // Calculate sizes of two subarrays
        int n1 = mid - left + 1;
        int n2 = right - mid;

        // Create temporary arrays
        int[] leftArr = new int[n1];
        int[] rightArr = new int[n2];

        // Copy data to temp arrays
        System.arraycopy(arr, left, leftArr, 0, n1);
        System.arraycopy(arr, mid + 1, rightArr, 0, n2);

        // Merge temp arrays back
        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) {
            if (leftArr[i] ⩽ rightArr[j]) {
                arr[k] = leftArr[i];
                i++;
            } else {
                arr[k] = rightArr[j];
                j++;
            }
            k++;
        }

        // Copy remaining elements
        while (i < n1) { arr[k] = leftArr[i]; i++; k++; }
        while (j < n2) { arr[k] = rightArr[j]; j++; k++; }
    }

    public static void main(String[] args) {
```

Quick Sort

Quick Sort — O(n log n) average, O(n^2) worst

Picks a 'pivot' element and partitions the array around the pivot such that elements smaller than the pivot are on the left, and greater elements are on the right. Recursively sorts the subarrays.

```
public class QuickSort {
    public static void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            int pivotIndex = partition(arr, low, high);

            quickSort(arr, low, pivotIndex - 1);   // Sort left
            quickSort(arr, pivotIndex + 1, high);  // Sort right
        }
    }

    private static int partition(int[] arr, int low, int high) {
        int pivot = arr[high];  // Choose last element as pivot
        int i = low - 1;        // Index of smaller element

        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++;
                swap(arr, i, j);
            }
        }

        swap(arr, i + 1, high);
        return i + 1;
    }

    private static void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};
        quickSort(arr, 0, arr.length - 1);
        System.out.println("Sorted: " + Arrays.toString(arr));
    }
}
```

Sorting Algorithm Comparison

Table 12-1: Sorting Algorithms Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case	Space	Stable
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No
Arrays.sort()	O(n log n)	O(n log n)	O(n log n)	O(log n)	Yes
Collections.sort()	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes

Which Sorting Algorithm to Use?

- Use `Arrays.sort()` or `Collections.sort()` for most cases (optimized TimSort)
- Use Quick Sort for in-place sorting of primitive arrays
- Use Merge Sort when stability is required
- Use Insertion Sort for small or nearly-sorted arrays
- Consider Bucket Sort or Counting Sort for integers with limited range

12.2 Searching Algorithms

Linear Search

Linear Search — O(n)

Sequentially checks each element until the target is found or the end is reached. Works on unsorted data.

```java
public class LinearSearch {
    // Returns index of target, or -1 if not found
    public static int linearSearch(int[] arr, int target) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                return i;
            }
        }
        return -1;
    }

    // Find all occurrences
    public static List<Integer> findAll(int[] arr, int target) {
        List<Integer> indices = new ArrayList<>();
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                indices.add(i);
            }
        }
        return indices;
    }
}
```

Binary Search

Binary Search — O(log n)

Finds the position of a target value within a **sorted** array by repeatedly dividing the search interval in half. Compare the target with the middle element and eliminate half of the remaining elements.

```
public class BinarySearch {
    // Iterative binary search
    public static int binarySearch(int[] arr, int target) {
        int left = 0;
        int right = arr.length - 1;

        while (left ⩽ right) {
            int mid = left + (right - left) / 2;  // Avoid overflow

            if (arr[mid] == target) {
                return mid;           // Found
            } else if (arr[mid] < target) {
                left = mid + 1;       // Search right half
            } else {
                right = mid - 1;      // Search left half
            }
        }
        return -1;  // Not found
    }

    // Recursive binary search
     public static int binarySearchRecursive(int[] arr, int target, int
left, int right) {
        if (left > right) return -1;

        int mid = left + (right - left) / 2;

        if (arr[mid] == target) return mid;
        if (arr[mid] < target) {
            return binarySearchRecursive(arr, target, mid + 1, right);
        }
        return binarySearchRecursive(arr, target, left, mid - 1);
    }

    // Find first occurrence
    public static int findFirst(int[] arr, int target) {
        int left = 0, right = arr.length - 1;
        int result = -1;

        while (left ⩽ right) {
            int mid = left + (right - left) / 2;

            if (arr[mid] == target) {
                result = mid;
                right = mid - 1;  // Continue searching left
            } else if (arr[mid] < target) {
```

Search Algorithm Comparison

Table 12-2: Searching Algorithms Comparison

Algorithm	Time Complexity	Space	Requirement
Linear Search	O(n)	O(1)	None
Binary Search	O(log n)	O(1)	Sorted array
Interpolation Search	O(log log n) avg	O(1)	Uniformly distributed sorted array
Hash Table	O(1) average	O(n)	Extra space for hash table

12.3 Recursion

> **Definition: Recursion**
>
> **Recursion** is a programming technique where a method calls itself to solve a problem by breaking it down into smaller, self-similar subproblems. A recursive solution requires a **base case** (termination condition) and a **recursive case**.

```
public class RecursionExamples {

    // Factorial: n! = n * (n-1)!
    public static long factorial(int n) {
        // Base case
        if (n ⩽ 1) return 1;
        // Recursive case
        return n * factorial(n - 1);
    }

    // Fibonacci: F(n) = F(n-1) + F(n-2)
    // Inefficient naive approach: O(2^n)
    public static long fibonacci(int n) {
        if (n ⩽ 1) return n;
        return fibonacci(n - 1) + fibonacci(n - 2);
    }

    // Efficient with memoization: O(n)
    public static long fibonacciMemo(int n, long[] memo) {
        if (n ⩽ 1) return n;
        if (memo[n] ≠ 0) return memo[n];
           memo[n] = fibonacciMemo(n - 1, memo) + fibonacciMemo(n - 2,
memo);
        return memo[n];
    }

    // Sum of array elements
    public static int arraySum(int[] arr, int index) {
        if (index ⩾ arr.length) return 0;
        return arr[index] + arraySum(arr, index + 1);
    }

    // Check if string is palindrome
    public static boolean isPalindrome(String s, int left, int right) {
        if (left ⩾ right) return true;
        if (s.charAt(left) ≠ s.charAt(right)) return false;
        return isPalindrome(s, left + 1, right - 1);
    }

    // Power calculation: x^n
    public static double power(double x, int n) {
        // Base case
        if (n == 0) return 1;
        if (n < 0) return 1 / power(x, -n);

        // Recursive case (optimized)
```

12.4 Dynamic Programming

> **Definition: Dynamic Programming**
>
> **Dynamic Programming (DP)** is an optimization technique used to solve complex problems by breaking them into overlapping subproblems, solving each subproblem only once, and storing the results for future use (memoization or tabulation).

```
public class DynamicProgramming {

    // Fibonacci using tabulation (bottom-up)
    // Time: O(n), Space: O(n) — can be optimized to O(1)
    public static long fibonacciDP(int n) {
        if (n ⩽ 1) return n;

        long[] dp = new long[n + 1];
        dp[0] = 0;
        dp[1] = 1;

        for (int i = 2; i ⩽ n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }

        return dp[n];
    }

    // Space-optimized Fibonacci
    public static long fibonacciOptimized(int n) {
        if (n ⩽ 1) return n;

        long prev2 = 0;  // F(i-2)
        long prev1 = 1;  // F(i-1)

        for (int i = 2; i ⩽ n; i++) {
            long current = prev1 + prev2;
            prev2 = prev1;
            prev1 = current;
        }

        return prev1;
    }

    // Coin Change: minimum coins to make amount
    // Time: O(amount * coins), Space: O(amount)
    public static int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);  // Initialize with infinity
        dp[0] = 0;  // 0 coins needed for amount 0

        for (int i = 1; i ⩽ amount; i++) {
            for (int coin : coins) {
                if (coin ⩽ i) {
                    dp[i] = Math.min(dp[i], dp[i - coin] + 1);
                }
```

Chapter 13

Multithreading and Concurrency

Multithreading allows a program to execute multiple threads concurrently, enabling better utilization of CPU resources and responsive applications.

13.1 Creating Threads

> **Definition: Thread**
>
> A **thread** is a lightweight process — the smallest unit of execution within a program. Multiple threads within the same process share the same memory space but execute independently.

Method 1: Extending Thread Class

```
public class MyThread extends Thread {
    @Override
    public void run() {
        for (int i = 1; i ⩽ 5; i++) {
             System.out.println(Thread.currentThread().getName() + ": " +
i);
            try {
                Thread.sleep(500);  // Sleep for 500ms
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted");
            }
        }
    }

    public static void main(String[] args) {
        MyThread thread1 = new MyThread();
        MyThread thread2 = new MyThread();

        thread1.setName("Thread-A");
        thread2.setName("Thread-B");

        thread1.start();  // Start thread (calls run() internally)
        thread2.start();

        System.out.println("Main thread continues ... ");
    }
}
```

Method 2: Implementing Runnable Interface

```
public class MyRunnable implements Runnable {
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(Thread.currentThread().getName() + ": " +
i);
        }
    }

    public static void main(String[] args) {
        // Using Runnable (preferred — supports multiple inheritance)
        Runnable task = new MyRunnable();

        Thread thread1 = new Thread(task, "Worker-1");
        Thread thread2 = new Thread(task, "Worker-2");

        thread1.start();
        thread2.start();

        // Lambda approach (Java 8+)
        Thread thread3 = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                System.out.println("Lambda-Thread: " + i);
            }
        });
        thread3.start();
    }
}
```

13.2 Thread Lifecycle

Table 13-1: Thread States

State	Description	Trigger
NEW	Thread created but not started	After `new Thread()`
RUNNABLE	Ready to run or running	After `start()` called
BLOCKED	Waiting for a monitor lock	Trying to enter synchronized block
WAITING	Indefinitely waiting	`wait()`, `join()` without timeout
TIMED_WAITING	Waiting for specified time	`sleep()`, `wait(timeout)`
TERMINATED	Execution completed	`run()` method finishes

13.3 Synchronization

> **Definition: Synchronization**
>
> **Synchronization** ensures that only one thread at a time can access a shared resource, preventing race conditions and ensuring thread safety. It is achieved using the `synchronized` keyword or locks.

```
public class Counter {
    private int count = 0;

    // Synchronized method — only one thread can execute at a time
    public synchronized void increment() {
        count++;
    }

    // Synchronized block — finer control
    public void decrement() {
        synchronized (this) {
            count--;
        }
    }

    public synchronized int getCount() {
        return count;
    }

    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();

        // Create multiple threads that increment
        Thread[] threads = new Thread[5];
        for (int i = 0; i < 5; i++) {
            threads[i] = new Thread(() -> {
                for (int j = 0; j < 1000; j++) {
                    counter.increment();
                }
            });
            threads[i].start();
        }

        // Wait for all threads to complete
        for (Thread t : threads) {
            t.join();
        }

        System.out.println("Count: " + counter.getCount());  // 5000
    }
}
```

```
import java.util.concurrent.locks.*;

public class ReentrantLockExample {
    private int count = 0;
    private final Lock lock = new ReentrantLock();
    private final Condition condition = lock.newCondition();

    public void increment() {
        lock.lock();  // Acquire lock
        try {
            count++;
            if (count >= 10) {
                condition.signalAll();  // Notify waiting threads
            }
        } finally {
            lock.unlock();  // Always release lock
        }
    }

    public void waitForCount() throws InterruptedException {
        lock.lock();
        try {
            while (count < 10) {
                condition.await();  // Wait until count >= 10
            }
            System.out.println("Count reached 10!");
        } finally {
            lock.unlock();
        }
    }
}
```

13.4 Executors Framework

```
import java.util.concurrent.*;

public class ExecutorsDemo {
    public static void main(String[] args) throws Exception {
        // 1. Fixed thread pool
        ExecutorService fixedPool = Executors.newFixedThreadPool(4);

        // Submit tasks
        for (int i = 0; i < 10; i++) {
            final int taskId = i;
            fixedPool.submit(() -> {
                System.out.println("Task " + taskId + " running on "
                    + Thread.currentThread().getName());
            });
        }
        fixedPool.shutdown();

        // 2. Cached thread pool
        ExecutorService cachedPool = Executors.newCachedThreadPool();

        // 3. Single thread executor
        ExecutorService single = Executors.newSingleThreadExecutor();

        // 4. Scheduled executor
                                ScheduledExecutorService    scheduler    =
Executors.newScheduledThreadPool(2);

        // Schedule once after delay
         scheduler.schedule(() -> System.out.println("Delayed task"), 5,
TimeUnit.SECONDS);

        // Schedule at fixed rate
        scheduler.scheduleAtFixedRate(
            () -> System.out.println("Repeating task"),
            0, 2, TimeUnit.SECONDS
        );

        // 5. Callable and Future (returns result)
        ExecutorService executor = Executors.newSingleThreadExecutor();

        Callable<Integer> task = () -> {
            Thread.sleep(1000);
            return 42;
        };

        Future<Integer> future = executor.submit(task);
```

Chapter 14

File I/O and Serialization

Java provides comprehensive APIs for reading from and writing to files, as well as for serializing objects to persistent storage.

14.1 File Class

> **Definition**
>
> The `java.io.File` class represents a file or directory path in the filesystem. It provides methods for file metadata operations but does not handle reading or writing file contents.

```
import java.io.File;
import java.io.IOException;

public class FileOperations {
    public static void main(String[] args) {
        File file = new File("test.txt");

        // File information
        System.out.println("Exists: " + file.exists());
        System.out.println("Name: " + file.getName());
        System.out.println("Path: " + file.getPath());
        System.out.println("Absolute: " + file.getAbsolutePath());
        System.out.println("Size: " + file.length() + " bytes");

        // File operations
        try {
            if (file.createNewFile()) {
                System.out.println("File created!");
            }
        } catch (IOException e) {
            e.printStackTrace();
        }

        // Directory operations
        File dir = new File("myFolder");
        if (dir.mkdir()) {
            System.out.println("Directory created!");
        }

        // List directory contents
        File currentDir = new File(".");
        String[] files = currentDir.list();
        if (files ≠ null) {
            for (String f : files) {
                System.out.println(f);
            }
        }

        // Recursive listing
        listFilesRecursively(currentDir, 0);
    }

    static void listFilesRecursively(File dir, int level) {
        File[] files = dir.listFiles();
        if (files == null) return;
```

14.2 Reading and Writing Files

Reading Files (Multiple Approaches)

```
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class FileReading {
    public static void main(String[] args) throws IOException {
        String filePath = "data.txt";

        // Method 1: BufferedReader (classic approach)
                  try  (BufferedReader  br  =  new  BufferedReader(new
FileReader(filePath))) {
            String line;
            while ((line = br.readLine()) ≠ null) {
                System.out.println(line);
            }
        }

        // Method 2: Files.readAllLines (Java 7+)
        List<String> allLines = Files.readAllLines(Path.of(filePath));

        // Method 3: Files.lines with Stream (Java 8+)
        try (Stream<String> lines = Files.lines(Path.of(filePath))) {
            lines.filter(line → line.contains("Java"))
                 .forEach(System.out::println);
        }

        // Method 4: Read entire file as String
        String content = Files.readString(Path.of(filePath));

        // Method 5: Scanner
        try (Scanner scanner = new Scanner(new File(filePath))) {
            while (scanner.hasNextLine()) {
                System.out.println(scanner.nextLine());
            }
        }
    }
}
```

Writing Files

```
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class FileWriting {
    public static void main(String[] args) throws IOException {
        // Method 1: BufferedWriter
                try (BufferedWriter writer = new BufferedWriter(new
FileWriter("output.txt"))) {
            writer.write("Hello, World!");
            writer.newLine();
            writer.write("Second line");
        }

        // Method 2: FileWriter with append mode
        try (FileWriter writer = new FileWriter("output.txt", true)) {
            writer.write("\nAppended line");
        }

        // Method 3: Files.write (Java 7+)
           List<String> lines = Arrays.asList("Line 1", "Line 2", "Line
3");
        Files.write(Path.of("output.txt"), lines);

        // Method 4: PrintWriter
        try (PrintWriter pw = new PrintWriter("output.txt")) {
            pw.println("Name: John");
            pw.printf("Age: %d%n", 25);
            pw.println("Score: " + 95.5);
        }

        // Method 5: Write string directly
        Files.writeString(Path.of("output.txt"), "Simple content");
    }
}
```

14.3 Serialization

> **Definition: Serialization**
>
> **Serialization** is the process of converting an object's state into a byte stream that can be stored or transmitted. **Deserialization** is the reverse process of reconstructing the object from the byte stream.

```
import java.io.*;

// Must implement Serializable to be serializable
public class Employee implements Serializable {
    // serialVersionUID ensures version compatibility
    private static final long serialVersionUID = 1L;

    private String name;
    private int age;
    private transient double salary;  // transient — not serialized
      private static String company = "Tech Corp";   // static — not
serialized

    public Employee(String name, int age, double salary) {
        this.name = name;
        this.age = age;
        this.salary = salary;
    }

    @Override
    public String toString() {
        return "Employee{name='" + name + "', age=" + age
            + ", salary=" + salary + "}";
    }

    // Serialize object
     public static void serialize(Employee emp, String filename) throws
IOException {
        try (ObjectOutputStream oos = new ObjectOutputStream(
                new FileOutputStream(filename))) {
            oos.writeObject(emp);
            System.out.println("Object serialized successfully!");
        }
    }

    // Deserialize object
    public static Employee deserialize(String filename)
            throws IOException, ClassNotFoundException {
        try (ObjectInputStream ois = new ObjectInputStream(
                new FileInputStream(filename))) {
            Employee emp = (Employee) ois.readObject();
            System.out.println("Object deserialized: " + emp);
            return emp;
        }
    }
```

Chapter 15

Best Practices and Design Patterns

Writing clean, maintainable, and efficient Java code requires following established best practices and design patterns. These principles make code easier to understand, test, and extend.

15.1 Coding Standards

Naming Conventions

Table 15-1: Java Naming Conventions

Type	Convention	Examples
Classes	PascalCase, noun	`CustomerOrder`, `HttpRequest`
Interfaces	PascalCase, adjective/noun	`Comparable`, `Serializable`
Methods	camelCase, verb-noun	`getCustomerName()`, `calculateTotal()`
Variables	camelCase	`firstName`, `accountBalance`
Constants	UPPER_SNAKE_CASE	`MAX_SIZE`, `PI`
Packages	lowercase, reversed domain	`com.company.project`

SOLID Principles

> **SOLID Principles**
>
> **SOLID** is an acronym for five design principles that help developers build maintainable and scalable software.

```
// S - Single Responsibility Principle
// A class should have only one reason to change

// BAD: Class has multiple responsibilities
class EmployeeBad {
    void calculatePay() { }
    void saveToDatabase() { }
    void generateReport() { }
}

// GOOD: Separate responsibilities
class Employee {
    private String name;
    private double salary;
    // Getters, setters, business logic only
}

class EmployeeRepository {
    void save(Employee emp) { /* Database logic */ }
}

class EmployeeReportGenerator {
     String generateReport(Employee emp) { /* Report logic */ return "";
}
}

// O - Open/Closed Principle
// Open for extension, closed for modification

interface DiscountStrategy {
    double calculateDiscount(double amount);
}

class RegularDiscount implements DiscountStrategy {
      public double calculateDiscount(double amount) { return amount *
0.1; }
}

class PremiumDiscount implements DiscountStrategy {
      public double calculateDiscount(double amount) { return amount *
0.2; }
}

class DiscountCalculator {
    double applyDiscount(double amount, DiscountStrategy strategy) {
        return amount - strategy.calculateDiscount(amount);
```

15.2 Common Design Patterns

Singleton Pattern

> **Singleton Pattern**
>
> Ensures a class has only one instance and provides a global point of access to it. Useful for shared resources like database connections, caches, and configuration.

```
// Thread-safe Singleton (Bill Pugh approach — preferred)
public class DatabaseConnection {
    // Private constructor prevents instantiation
    private DatabaseConnection() {
        System.out.println("Database connection established");
    }

    // Static inner class — loaded only when referenced
    private static class SingletonHelper {
            private static final DatabaseConnection INSTANCE = new
DatabaseConnection();
    }

    public static DatabaseConnection getInstance() {
        return SingletonHelper.INSTANCE;
    }

    public void executeQuery(String sql) {
        System.out.println("Executing: " + sql);
    }
}

// Enum Singleton (most robust, prevents reflection attacks)
public enum ConfigManager {
    INSTANCE;

    private String config;

    public void setConfig(String config) { this.config = config; }
    public String getConfig() { return config; }
}
```

Factory Pattern

```
// Product interface
interface Notification {
    void send(String message);
}

// Concrete products
class EmailNotification implements Notification {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}

class SMSNotification implements Notification {
    public void send(String message) {
        System.out.println("SMS: " + message);
    }
}

class PushNotification implements Notification {
    public void send(String message) {
        System.out.println("Push: " + message);
    }
}

// Factory class
class NotificationFactory {
    public Notification createNotification(String type) {
        return switch (type.toLowerCase()) {
            case "email" -> new EmailNotification();
            case "sms" -> new SMSNotification();
            case "push" -> new PushNotification();
             default -> throw new IllegalArgumentException("Unknown type:
" + type);
        };
    }
}

// Usage
public class FactoryDemo {
    public static void main(String[] args) {
        NotificationFactory factory = new NotificationFactory();

        Notification email = factory.createNotification("email");
        email.send("Hello!");  // Email: Hello!

        Notification sms = factory.createNotification("sms");
```

Observer Pattern

```
import java.util.*;

// Observer interface
interface Observer {
    void update(String event);
}

// Subject (Observable)
class NewsAgency {
    private List<Observer> observers = new ArrayList<>();
    private String latestNews;

    public void attach(Observer observer) {
        observers.add(observer);
    }

    public void detach(Observer observer) {
        observers.remove(observer);
    }

    public void setNews(String news) {
        this.latestNews = news;
        notifyObservers();
    }

    private void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(latestNews);
        }
    }
}

// Concrete observers
class NewsChannel implements Observer {
    private String name;

    public NewsChannel(String name) { this.name = name; }

    @Override
    public void update(String event) {
        System.out.println(name + " received: " + event);
    }
}

// Usage
public class ObserverDemo {
```

Builder Pattern

```
// Product
public class Computer {
    private String cpu;
    private int ram;
    private int storage;
    private boolean hasGPU;
    private boolean hasBluetooth;

    private Computer(Builder builder) {
        this.cpu = builder.cpu;
        this.ram = builder.ram;
        this.storage = builder.storage;
        this.hasGPU = builder.hasGPU;
        this.hasBluetooth = builder.hasBluetooth;
    }

    // Builder class
    public static class Builder {
        // Required parameters
        private String cpu;
        private int ram;

        // Optional parameters with defaults
        private int storage = 256;
        private boolean hasGPU = false;
        private boolean hasBluetooth = false;

        public Builder(String cpu, int ram) {
            this.cpu = cpu;
            this.ram = ram;
        }

        public Builder storage(int val) { storage = val; return this; }
         public Builder hasGPU(boolean val) { hasGPU = val; return this;
}
          public Builder hasBluetooth(boolean val) { hasBluetooth = val;
return this; }

        public Computer build() {
            return new Computer(this);
        }
    }

    @Override
    public String toString() {
          return String.format("Computer[CPU=%s, RAM=%dGB, Storage=%dGB,
```

15.3 Memory Management

> **Java Memory Model**
>
> Java memory is divided into several regions: **Heap** (objects and arrays), **Stack** (method frames and local variables), **Metaspace** (class metadata), **Code Cache** (JIT-compiled code), and **Direct Buffers** (NIO).

```
public class MemoryManagement {

    // 1. Avoid creating unnecessary objects
    public void efficientStringConcat() {
        // BAD: Creates many intermediate String objects
        String result = "";
        for (int i = 0; i < 100; i++) {
            result += i;  // Creates new String each iteration
        }

        // GOOD: Use StringBuilder
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 100; i++) {
            sb.append(i);
        }
        String result2 = sb.toString();
    }

    // 2. Use primitive types when possible
    public void preferPrimitives() {
        // BAD: Uses more memory
        Integer boxed = 100;  // Object overhead

        // GOOD: More efficient
        int primitive = 100;

        // Use primitive streams for large data
        int[] numbers = {1, 2, 3, 4, 5};
            int sum = Arrays.stream(numbers).sum();   // IntStream, not
Stream<Integer>
    }

    // 3. Nullify references when done (rarely needed)
    public void cleanup() {
        Object largeObject = new Object();
        // ... use largeObject ...
         largeObject = null;  // Help GC if object lives in a long-lived
scope
    }

    // 4. Use try-with-resources
    public void properResourceManagement() throws IOException {
        // BAD: Resource may not be closed
                    BufferedReader reader = new BufferedReader(new
FileReader("file.txt"));
        String line = reader.readLine();
```

Performance Tips

- Use `StringBuilder` for string concatenation in loops
- Prefer primitives over wrapper classes where possible
- Specify initial capacity for `ArrayList` and `HashMap` when known
- Use enhanced for-each loop instead of indexed loops for collections
- Consider lazy initialization for expensive objects
- Use `==` for enum comparisons, not `.equals()`
- Prefer `Arrays.copyOf()` over manual array copying
- Use `System.arraycopy()` for bulk array operations
- Avoid calling methods in loop termination conditions
- Use `final` for variables that don't change — enables compiler optimizations